

Predicting Traffic Accident Severity in Brazil: A Multiclass Machine Learning Approach with Imbalance Handling

Marek Martinak
Student ID: D00250465
MSc Data Analytics
Dundalk Institute of Technology

June 2026

Abstract

Records show that there are approximately 33,000 fatalities each year in Brazil as a result of traffic accidents, making this the tenth leading cause of death and contributing 4% of GDP in economic cost (approximately 220 billion reais). Therefore, policy makers and stakeholders need a tool that can reduce the number of severe traffic accidents and their casualties. This requires understanding what causes severe accidents. Prior studies mainly focused on binary classification (lethal, non-lethal), used only SMOTE or no imbalance handling technique, and omitted explainability analysis at the country level. The Brazilian PRF dataset (approximately 463,000 records, 2017-2023) was used to compare six classifiers. The top 3 classifiers were selected to be combined with seven imbalance methods. Optuna tuning was applied for the top 3 combinations. The best combination, XGBoost with RandomOver, was analysed with TreeSHAP to provide feature importance for all three severity classes. The final model was able to improve macro-F1 from 0.28 (DummyClassifier) to 0.52. It was also able to more than double the fatal-class recall from 22.4% (DecisionTree baseline) to 50.1%. TreeSHAP analysis for global ranking identified that the feature Accident_Type placed in the first position (contains 3x higher importance than the second position). This study demonstrates how simple resampling techniques (RandomOver, ClassWeight) can outperform sophisticated SMOTE on OHE (One-Hot Encoding) high-cardinality data. This finding can be applied to other studies that utilise similar features with high cardinality data. Future work could explore different imbalance handling techniques (SMOTE-NC), real-time risk forecasting, or testing on newer PRF data (2024 - 2025). With this direction, it would be possible to create a tool that can identify high-risk highway segments, prioritise emergency infrastructure, and target specific regions.

Contents

1	Background and Introduction	4
1.1	Why this matters in Data Analytics	4
1.2	Problem definition & Project relevance	4
1.3	Research Questions	4
1.4	Hypothesis	4
1.5	Project scope and approach	5
1.6	Report Structure	5
2	Literature Review	5
2.1	Traffic Accident Severity Classification	5
2.2	Class Imbalance Methods	6
2.3	Multi-class vs Binary Classification	7
2.4	Explainability and Feature Importance	8
2.5	Tree-based Models on Tabular Data	8
2.6	Gaps and Contribution	8
3	Data Exploration and Methodology	9
3.1	Data Collection	9
3.2	Ethical Approval	9
3.3	Data Description	9
3.4	Data Cleaning	11
3.5	Feature Engineering	13
3.6	Exploratory Data Analysis	14
3.6.1	Target Variable Distribution	14
3.6.2	Geographic Patterns	15
3.6.3	Temporal Patterns	17
3.6.4	Accident Causes	18
3.7	Phase 1 — Data Preparation and Evaluation Module	19
3.8	Phase 2 — Six-Classifer Baseline	20
3.9	Phase 3 — Imbalance Handling Experiments	21
3.10	Phase 4 — Hyperparameter Tuning	21
3.11	Phase 5 — TreeSHAP Explainability	22
3.12	Metrics and Evaluation	24
4	Results and Discussion	25
4.1	Phase 2 Baseline Results	25
4.2	Phase 3 Imbalance-Handling Results	26
4.3	Phase 4 Hyperparameter Tuning Results	28
4.4	Final Test-Set Evaluation	30
4.5	SHAP Feature Importance Findings	32

5	Conclusions and Future Work	35
5.1	Summary of Contributions	35
5.2	Answers to Research Questions	36
5.3	Limitations	37
5.4	Future Work	38
A	Project Links	40
B	Tools	40
B.1	Development Environment	40
B.2	Python Libraries	40
B.3	Writing and Translation Tools	41
C	Plots	41
C.1	Optuna Diagnostic Plots — Non-Winning Studies	41
C.2	SHAP Plots	43
D	Prompts for Generative AI	46

1 Background and Introduction

In today's world, vehicle transportation is almost indispensable. Nevertheless, in Brazil, it was confirmed to be the tenth most common cause of death, which annually has been confirmed to have around 33,000 deaths (Alves et al. 2021). The cost of traffic accidents in Brazil is around 220 billion reais per year, which accounts for up to 4% of Brazil's GDP (Nunes & Silva 2021).

1.1 Why this matters in Data Analytics

Accident severity prediction is a broad and well-studied field. The Brazilian PRF accident dataset contains around 500,000 data points, which should be capable of performing meaningful prediction models. Practical results could also contribute to better resource allocation and achieving a higher success rate in saving lives.

1.2 Problem definition & Project relevance

Most of the previous studies focused on binary classification and acknowledged class imbalance or only performed one technique (Scholz et al. 2024, Gabardo et al. 2026). Binary classification loses important information about injured cases by only addressing two variables (lethal/non-lethal). By properly distinguishing three severity levels, this approach can provide better emergency response allocation or resource planning. However, the challenges arise when this approach is chosen. Multi-classification exposes a strong class imbalance, which requires systematic imbalance handling techniques, because it is impractical to rely on a single default method. The standard metric accuracy is insufficient for evaluating the model when dealing with imbalanced data, because a model predicting only the majority class can still achieve high accuracy without learning any meaningful pattern. Therefore, this project aims to handle three severity classes: "With dead victims", "With injured victims", "Without victims" and apply several techniques to handle the imbalanced data with proper metrics to evaluate the model prediction and performance.

1.3 Research Questions

This study addresses three research questions:

1. RQ1: When it comes to predicting the multiclass severity of accidents using macro-F1 and per-class metrics, can tree-based classifiers (Decision Tree, Random Forest, XGBoost, LightGBM, and CatBoost) outperform a Logistic Regression baseline?
2. RQ2: How does handling class imbalance affect the performance of these models, and how do the main methods of class imbalance handling compare?
3. RQ3: Which features are most important in predicting traffic accident severity?

1.4 Hypothesis

The most effective classification approach in predicting accident severity will be tree-based rather than linear. The imbalance handling methods will improve macro-F1 compared to baseline

models without them. SHAP will select and identify the dominant predictors contributing to accident severity. Per-class explainability may differ across severity classes.

1.5 Project scope and approach

This dissertation is the final project for the Master’s degree in Data Analytics at Dundalk Institute of Technology. It is the culmination of several months’ work, combining data analysis, programming and an understanding of the results. Altogether, this project has been divided into 6 phases: data preparation, six-classifier baseline comparison, systematic imbalance-handling experiments, hyperparameter tuning via Optuna, TreeSHAP explainability and final documentation. The primary metric is Macro-F1 and the secondary is MCC (Matthews Correlation Coefficient), both suitable for imbalanced multiclass classification (justified in Section 2). The list of software tools, libraries, and versions is provided in Appendix B. Generative AI was used to assist with code per DkIT policy, and all prompts are documented in Appendix D.

1.6 Report Structure

- Section 2 reviews prior work on accident severity classification, imbalance handling, multiclass vs binary approaches, tree-based models for tabular data, and identifies gaps for further research.
- Section 3 describes the Brazilian PRF dataset, cleaning, feature engineering, EDA, and the six-phase methodology.
- Section 4 presents baseline model comparison, imbalance-handling experiments, hyperparameter tuning results, final test-set evaluation, and TreeSHAP feature importance findings.
- Section 5 answers each research question directly, acknowledges limitations, and outlines future work.

2 Literature Review

2.1 Traffic Accident Severity Classification

When it comes to predicting the severity of an accident, it is important to consider which methods to use. Statistical models, like logit, probit, ordered probit, and multinomial logit, were used before as the most conventional approach to crash-severity analysis, but recent research papers and tests discovered that machine learning models can outperform them on predictive accuracy (Zhang et al. 2018). Machine learning models are able to handle non-linear interactions and high-dimensional categorical features, which makes them more ideal for predictions than statistical models.

Machine learning models predicting accident severity can be split into two categories. Supervised (learns from labels) or unsupervised (finds patterns without labels), where the standard approach is to use supervised for this kind of topic. Deep learning models are commonly used

on image data, rather than tabular. Therefore, the classical supervised machine learning models are the main topic to be used on severity accidents data.

Zhang et al. (2018) primarily compared two statistical models for crash severity prediction with four machine learning models. Although all machine learning models achieved better results, they suffered from overfitting. This did not occur with the statistical models, but their results for minority prediction was 0%, which is insufficient. Zhang et al. (2018) concluded that higher accuracy doesn't mean trustworthy insights, because machine learning models can disagree with each other on variable importance and conclusion. However, this can provide further discussion.

Other studies Lee et al. (2021) predicted the severity of crashes and identify possible risk factors (human, vehicle, road, environmental, etc.) while using only a single CART decision tree. Data from the years 2004 - 2017 for around 170,000 US city crashes were used to predict the final four severity levels. While achieving a prominent 65.78% accuracy, the overall result led to a finding that the model failed to predict any fatal injuries (0% precision and 0% recall).

Labib et al. (2019) were able to achieve 80% accuracy on four-class severity. However, the dataset consists primarily of data related to fatal accidents, which means that the main results will pertain more to the prediction of the majority class than to performance on the minority class. This is a class imbalance issue, which can lead to bias when implemented.

Two recent studies targeted the Brazilian PRF dataset and applied machine learning models to predict accident severity. Scholz et al. (2024) performed binary predictions on five classifiers, such as Decision Tree, Random Forest, KNN, Naive Bayes, and MLP, on data from the year 2017 to 2022 on the Brazilian PRF dataset, enhanced by including vehicle prices as well, while stating the limitation of not performing any imbalance technique, which is a problem for prediction of minority classes. Meanwhile, Gabardo et al. (2026) applied SMOTE (Synthetic Minority Over-sampling Technique), which creates synthetic samples by interpolation between existing data points rather than duplicating minority-class instances. This was used for three classifiers (RF, KNN, MLP), but its limitation lies in the fact that it focused on only one region (southern Brazil) and was used only for binary classification.

Outside the Brazilian context, Ahmed et al. (2023) demonstrated that meaningful metrics are possible to achieve while focusing on using techniques such as SMOTE oversampling, ensemble methods and SHAP for explainability, which leads to a reliable and trustworthy implementation of the results.

These findings can be considered as a methodological stepping stone for this study, which aims to fill the gaps of previous studies by predicting severity across multiple classes using SMOTE oversampling technique, focusing on the entire country rather than just a specific region.

2.2 Class Imbalance Methods

Studies to this point confirm that, when it comes to predicting performance, it is necessary to use appropriate techniques to address class imbalance. Disadvantages of imbalance data can lead to meaningless accuracy, models predict majority classes, per-class metrics collapse on the minority class. The approaches to handle imbalanced data are (Narwane & Sawarkar 2019):

- Data-level - modify the training set (over/under-sampling)
- Algorithm-level - modify the classifier itself
- Cost-sensitive - penalise minority-class errors more heavily
- Ensemble-based - combine multiple weak learners with imbalance-aware schemes

One of the foundational data-level methods is SMOTE, which helps the model develop a better general understanding of decisions, rather than simply memorising recurring events. SMOTE is a technique to use on imbalanced data for machine learning models to identify fraud detection, medical diagnosis, and crash severity prediction ([Chawla et al. 2002](#)).

Although SMOTE is an important technique, it treats all minority classes equally, even though some of them are easier for a model to learn. However, ADASYN (Adaptive Synthetic) is able to generate synthetic samples for minority classes like SMOTE, but targets specific areas, which are harder to learn, thereby improving the model’s decision boundary. A limitation of using ADASYN is reduced precision, but in return, recall is increased ([He et al. 2008](#)).

To handle noise and class overlap, the hybrid methods such as SMOTE- Tomek and SMOTE-ENN exist. This cleans up ambiguous or noisy samples and enhances performance ([Narwane & Sawarkar 2019](#)).

[Tanha et al. \(2020\)](#) performed a benchmark on 19 multi-class datasets to compare methods on imbalanced data. It was discovered that SMOTEBoost consistently outperformed RUSBoost across the benchmark and native multi-class boosters, such as CatBoost and LightGBM, outperformed binary-decomposition OVO (One-Versus-One) approaches.

This subsection of literature points out the gaps in other researchers’ papers for crash-severity studies, where most of them do not handle the imbalanced data and are focused on binary classification rather than on multiple classifications, which is under research. This also confirms the methodological approach of this dissertation, which favours the SMOTE method over the RUS method, compares different kinds of imbalanced handling techniques and highlights the arguments for choosing multiclass classification over binary classification, which is further elaborated in the following section.

2.3 Multi-class vs Binary Classification

Previous studies on the severity of traffic accidents have focused on the use of a binary severity classification ([Scholz et al. 2024](#), [Gabardo et al. 2026](#)). Although predicting accurate results is essential, binary classification loses important information that could aid in emergency response, resource planning, and policy-making, enabling an appropriate response based on multiple classes, which is more suitable for real-life world scenarios. The difficulty with multi-classification problems comes with handling imbalanced data. Minority classes are more difficult to handle, multi-class imbalance receives less attention from the model and receives less research attention than binary classification ([Narwane & Sawarkar 2019](#)). Despite the increased difficulty of performing a multi-class classification model, there is evidence that native multi-class boosting outperforms binary-decomposition (OvO/OvR) approaches on imbalanced data ([Tanha et al. 2020](#)). There is a visible natural order in terms of severity. The previous empirical work

discovered that using a nominal multi-class model outperformed ordinal models, which leads to the conclusion of using nominal models, even though the data are ordinal (Zhang et al. 2018). Based on these findings, a methodological approach is being developed that focuses on the three-category nominal problem, with the challenge of dealing with data imbalance being addressed.

2.4 Explainability and Feature Importance

Simple explainability is important to offer for stakeholders in order to clearly define trust, accountability, verifiability, and ethical standards. It is necessary to be able to understand the model prediction. Models with higher accuracy and black-box systems are less valuable than explainable models with less accuracy. The insight is a key element to understand. Without it, unknown errors can be created and can undermine the credibility of the parties involved. Therefore, to be able to assign value to each feature for each individual prediction, SHAP (SHapley Additive exPlanations) is going to be used, which is a method to explain the output of any machine learning model. LIME (Local Interpretable Model-Agnostic Explanations), which is also a method to explain the output of any machine learning model and raw feature importance do not properly guarantee satisfaction of each desirable property, such as local accuracy, missingness, and consistency (Lundberg & Lee 2017). The LIME method allows only local interpretability, whereas the SHAP method can provide both local explanations and global feature rankings, which is more beneficial for answering research question 3. Ahmed et al. (2023) and Scholz et al. (2024) used SHAP to find the most appropriate predictor in their studies, which also supports the decision of using it on tree-based models.

Therefore, this leads to the conclusion to use SHAP for identifying the important predictors for accident severity data, revealing non-obvious patterns and providing proper insight for stakeholders or interested parties.

2.5 Tree-based Models on Tabular Data

Deep learning or neural network models are becoming a standard approach in many fields. Shwartz-Ziv & Armon (2022) was capable of proving that machine learning algorithms for structured data, such as XGBoost, are capable of dominating 8 out of 11 datasets against four "deep tabular" models (TabNet, NODE, DNF-Net, 1D-CNN). Each deep learning model was capable of performing well on its individual paper, but was unable to adapt to other possibilities. Grinsztajn et al. (2022) findings led to the conclusion that tree-based models are the most appropriate to use on medium-sized datasets based on 45 benchmark datasets testing. This led to the decision to use Random Forest, XGBoost, LightGBM, and CatBoost as the main models with logistic regression as the baseline model for performance evaluation, thereby achieving faster results.

2.6 Gaps and Contribution

This literature review addresses three gaps that this dissertation intends to fill.

- Multi-class on Brazilian PRF data

Previous empirical studies have focused on identifying binary classifications. However, they did not address severity as a problem for multidimensional classification, but merely mentioned it as a topic for future research (Scholz et al. 2024, Gabardo et al. 2026).

- Systematic imbalance-method comparison

Prior work tends to ignore, acknowledge or only apply one method of handling imbalance data (Ahmed et al. 2023, Gabardo et al. 2026, Lee et al. 2021, Labib et al. 2019). It lacks comparison of using multiple methods across three classifiers, which this dissertation intends to fill (Tanha et al. 2020, Narwane & Sawarkar 2019).

- Modern explainability on PRF data

This paper focuses on the TreeSHAP algorithm, a specific variant of the SHAP algorithm that is capable of achieving the best performance for multi-class models on the PRF national dataset, which other studies did not elaborate on or investigate (Scholz et al. 2024, Gabardo et al. 2026).

By answering the research questions, this paper addresses the gaps that the literature review established.

3 Data Exploration and Methodology

3.1 Data Collection

The data were collected by the Brazilian Federal Highway Police (PRF) (LippoM 2024). They are publicly available on the Kaggle repository. This study covered a period of seven years of data collection. More specifically, from 2017 to 2023, which contains 463,152 records. All the information about the traffic accidents is processed annually, filled in CSV format and merged together. This process occurred in each state, meaning that none were excluded. The data is geographically imbalanced because there is a higher incidence of accidents in some states versus others, particularly those with higher populations and urbanisation. This should be taken into account, as this work focused only on class imbalance for the target variable rather than geographical distribution.

3.2 Ethical Approval

The data were collected by the Brazilian government and are publicly available. The dataset on Kaggle is fully anonymised and does not present any personally identifiable information. These factors prevent unethical behaviour and lead to the conclusion that no user approval is required. This work acknowledges the use of Brazilian open government data for machine learning purposes, academic reuse, analysis, and publication.

3.3 Data Description

As mentioned in the literature review, this dataset is considered medium-sized. It consists of 463,152 rows and contains 27 columns, which were collected from 01/01/2017 to 20/08/2023.

Table 1: Description of the 27 raw columns in the PRF dataset.

Column	Type	Description
inverse_data	str	Date of the accident (originally in reversed format)
week_day	str	Day of the week (Monday–Sunday)
hour	str	Hour at which the accident occurred
weather_timestamp	str	Phase of the day (sunrise, day, sunset, night)
state	str	Brazilian state of occurrence (e.g. SP, RJ, MA)
road_id	float64	Federal highway identifier (e.g. 116, 101)
km	str	Kilometre marker along the highway
city	str	City where the accident occurred
road_direction	str	Direction relative to the kilometre marker (increasing / decreasing)
latitude	float64	GPS latitude of the accident
longitude	float64	GPS longitude of the accident
regional	str	PRF regional administrative district
police_station	str	PRF police station that recorded the accident
cause_of_accident	str	Reported cause of the accident (80+ raw categories)
type_of_accident	str	Collision type
weather_condition	str	Weather at time of accident
road_type	str	Road layout type (single lane, double lane, etc.)
road_delineation	str	Lane marking and delineation condition
people	int64	Total number of people involved
deaths	int64	Number of fatalities
slightly_injured	int64	Number of people with minor injuries
severely_injured	int64	Number of people with serious injuries
uninjured	int64	Number of people unhurt
ignored	int64	Number of people whose condition was not reported
total_injured	int64	Sum of slight and severe injuries
vehicles_involved	int64	Number of vehicles involved
victims_condition	str	Severity classification: Without / With injured / With dead victims

The columns can be labelled and categorised for better clarification and description. This will result in 5 categories.

Table 2: Raw columns of the PRF dataset grouped by category.

Category	Count	Columns
Temporal	4	<code>inverse_data</code> , <code>week_day</code> , <code>hour</code> , <code>weather_timestamp</code>
Location	9	<code>state</code> , <code>road_id</code> , <code>km</code> , <code>city</code> , <code>road_direction</code> , <code>latitude</code> , <code>longitude</code> , <code>regional</code> , <code>police_station</code>
Accident Circumstance	5	<code>cause_of_accident</code> , <code>type_of_accident</code> , <code>wheather_condition</code> , <code>road_type</code> , <code>road_delineation</code>
Outcome	8	<code>people</code> , <code>deaths</code> , <code>slightly_injured</code> , <code>severely_injured</code> , <code>uninjured</code> , <code>ignored</code> , <code>total_injured</code> , <code>vehicles_involved</code>
Target	1	<code>victims_condition</code>
Total	27	

The target variable for this project is *victims_condition*, which is a three-class categorical variable (Without victims, With injured victims, With dead victims). The category With injured victims has the largest share of this target variable, representing approximately 71% of the total data. In contrast, the smallest category is “With dead victims”, which represents approximately 6.8% of the total data. This should provide a general understanding of how imbalanced the target variable is. The attributes in category *Outcome* are unusable because they contain variables after the accident happened, which is inadequate for model prediction.

3.4 Data Cleaning

Data cleaning is the first step toward verifying the reliability of the data and, if necessary, improving it. As seen in Table 1, it is apparent that the column labels are insufficient. For example, `inverse_data`, `weather_timestamp`, and `wheather_condition` contain grammatical errors and ambiguities. Therefore, the step was taken to rename all columns for greater clarity and readability. The result is shown in Table 3.

During the examination of data. It was decided to combine the Date and Hour column. A single column, Datetime, was created, and the original Date and Hour columns were dropped. This can prevent redundancy and should be easier to use. This column was also moved to the front of the dataset for better readability. The column Road_Km has a data type string, which is inappropriate. Machine learning models are not capable of processing text. Therefore, the type was changed to be numeric.

The following step was to handle missing values. It was discovered that columns with missing values are Road_ID, Road_Km, Regional, Police_Station. The column with the smallest amount of missing values is Regional, which contains 10 of them. It was decided to drop those rows, because the amount would have no effect on future model performance. Assuming that the Road_ID and Road_Km variables are connected, which was based on the number of missing values (990), the step to nullify this issue was to fill the missing values with NO_ROAD for Road_ID and -1 for Road_Km. Furthermore, it was decided to create a column (Missing_Road_Data) which will directly show the model that these sections contain missing values. The last column with missing values was Police_Station. The same approach was taken. The missing values were filled with UNKNOWN, and the Missing_Police_Station column was cre-

ated for the model to identify the missing values.

One of the last steps to complete the Data cleaning was to check for duplicates. 9 duplicates were found, and based on their content, it was decided to remove them. None of them showed anything significant or unusual that would have led to the decision to keep them.

The last step was to check the outliers. The result is shown in Table 4. Only Latitude and Longitude outliers were dropped, as those outliers were incorrect coordinates. The other outliers serve as examples of rare but valid results that represent unusual cases. It should be noted that columns with outliers were excluded, as they are not suitable for model prediction because these features are outcome-type and not beneficial.

This way, the data cleaning was completed, and the work can continue for the next phase.

Table 3: Column renaming applied to the raw PRF dataset.

Original	Renamed	Reason
inverse_data	Date	Misleading name, standardised
week_day	Day	Simplified
hour	Hour	Casing
state	State	Casing
road_id	Road_ID	Casing
km	Road_Km	Clarified meaning
city	City	Casing
cause_of_accident	Accident_Cause	Reordered for consistency
type_of_accident	Accident_Type	Reordered for consistency
victims_condition	Victims_Condition	Casing
weather_timestamp	Day_Phase	Misleading: held day phase, not a timestamp
road_direction	Km_Direction	Clarified meaning
wheather_condition	Weather_Condition	Corrected spelling typo
road_type	Road_Type	Casing
road_delineation	Road_Delineation	Casing
people	People	Casing
deaths	Deaths	Casing
slightly_injured	Slightly_Injured	Casing
severely_injured	Severely_Injured	Casing
uninjured	Uninjured	Casing
ignored	Ignored	Casing
total_injured	Total_Injured	Casing
vehicles_involved	Vehicles_Involved	Casing
latitude	Latitude	Casing
longitude	Longitude	Casing
regional	Regional	Casing
police_station	Police_Station	Casing

Table 4: IQR-based outlier detection across numeric columns.

Column	Outliers	Share
People	8,586	1.9%
Deaths	31,344	6.8%
Slightly_Injured	20,283	4.4%
Severely_Injured	96,750	20.9%
Uninjured	37,195	8.0%
Ignored	57,240	12.4%
Total_Injured	30,484	6.6%
Vehicles_Involved	8,573	1.9%
Road_Km	4,356	0.9%
Invalid coordinates (dropped)	50	0.01%

3.5 Feature Engineering

During Exploratory Data Analysis (EDA), 11 feature columns were created for a proper understanding of temporal, geographic, and categorical structure, which were not available to use in the original data.

Temporal features were created with the help of a column called Datetime. These features help to analyse the time pattern of accidents. One feature that needs to be explained is Day_Type. One of the topics considered for exploration during EDA was whether or not Saturdays and Sundays exhibit a higher incidence of fatalities versus weekdays. The results led to the conclusion that Sundays are deadlier than Saturdays, and therefore, the Day_Type type was created to address this issue.

The Accident_Cause column has over 80 categories, which are not equally split. This was solved by merging certain categories to create a column with only 11 broader categories. This creates a low cardinality categorical predictor, which should be more useful.

Similarly, the Region_BR was created with five official macro regions from the State column. States were assigned to official regions based on the Brazilian Institute of Geography and Statistics (IBGE). This helped reveal that the North and Northeast are deadlier than other regions.

Lastly, as mentioned before, the Missing_Road_Data and Missing_Police_Station columns were created to help the model identify missing values. Possibly, the missing data are not random and could give important information about reporting quality and accident context.

The overall results are visible in Table 5. This table summarises the features described above.

Table 5: Engineered features derived from the raw PRF dataset.

Feature	Derived from	Type	Purpose
Datetime	Date + Hour	timestamp	Unifies date and hour into a single field
Hour	Datetime	integer (0–23)	Captures hour of day pattern
Month	Datetime	integer (1–12)	Captures seasonality
Weekday	Datetime	categorical (7)	Day name, replaces the raw Day column
Is_Weekend	Datetime	boolean	Quick binary weekend indicator
Day_Type	Datetime	categorical (3)	Weekday / Saturday / Sunday splits the weekend because Sat and Sun differ
Time_Period	Hour	categorical (4)	Morning / Afternoon / Evening / Night
Cause_Group	Accident_Cause	categorical (11)	Collapses 80+ causes into broader categories
Region_BR	State	categorical (5)	Maps 27 states to 5 IBGE macro regions
Missing_Road_Data	Road_ID	boolean	Preserves missingness signal for Road_ID
Missing_Police_Station	Police_Station	boolean	Preserves missingness signal for Police_Station

3.6 Exploratory Data Analysis

The EDA is trying to explore the clean dataset while inspecting and interpreting the meaning of the plots and results. After cleaning, the dataset contains 463,133 records.

3.6.1 Target Variable Distribution

The target variable exhibits a highly imbalanced distribution. As mentioned previously, the minority class is the one with dead victims, which contains around 6.8% of the overall data, and the most common class is the one with injured victims, which contains 71.7%. If a naive classifier were used, the one that would ignore the features input, it would predict accuracy to be 71.7%, which is correct by the total share of accidents, but it would lack proper understanding. Therefore, this is the main motive for the second research question. It also illustrates that accuracy is not the proper metric to use in this case. Therefore, the per-class metrics such as F1, Macro-F1 and Matthews Correlation Coefficient (MCC) will be used to compare the results as well. [Tanha et al. \(2020\)](#) mentioned that MCC is the best metric to use on multiclass imbalanced data.

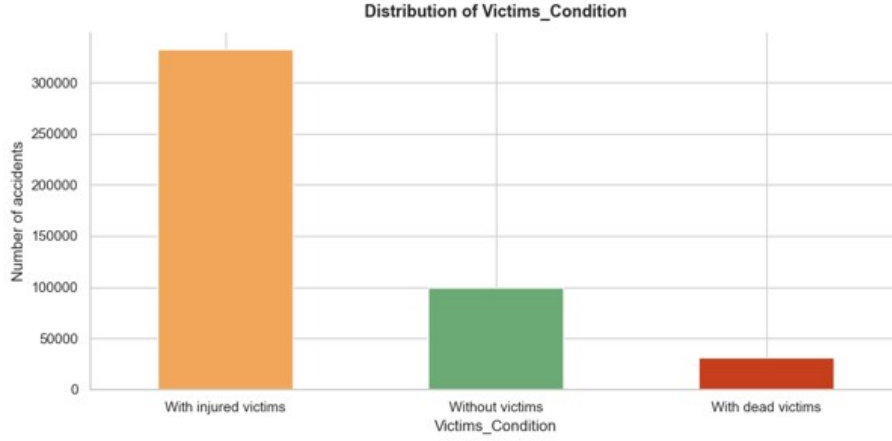


Figure 1: Distribution of the target variable `Victims_Condition`

3.6.2 Geographic Patterns

During the data examination, there was a step to aggregate states into broader categories. Macro-regions were created to find visible patterns, which would be invisible with 27 states. When each record was mapped based on the 5 IBGE macro-regions, it was discovered that Accidents in the northeast tend to exhibit double the fatality rate compared to those in the south. This result leads to the assumption that most likely the `Region_BR` column will be an important predictor. This connects to research question 3, where SHAP will be used for the analysis. [Scholz et al. \(2024\)](#) found that the region category is one of the most important predictors for the PRF dataset. This should confirm that the hypothesis is correct and that it will prove to be true.

Table 6: Fatality and injury profile across the five Brazilian macro-regions.

Region_BR	Accidents	Injured share	Fatal share
Northeast	98,076	68.4%	10.2%
North	26,182	70.1%	8.9%
Centre-West	56,516	69.6%	7.0%
Southeast	141,603	74.8%	5.7%
South	140,756	72.1%	5.1%

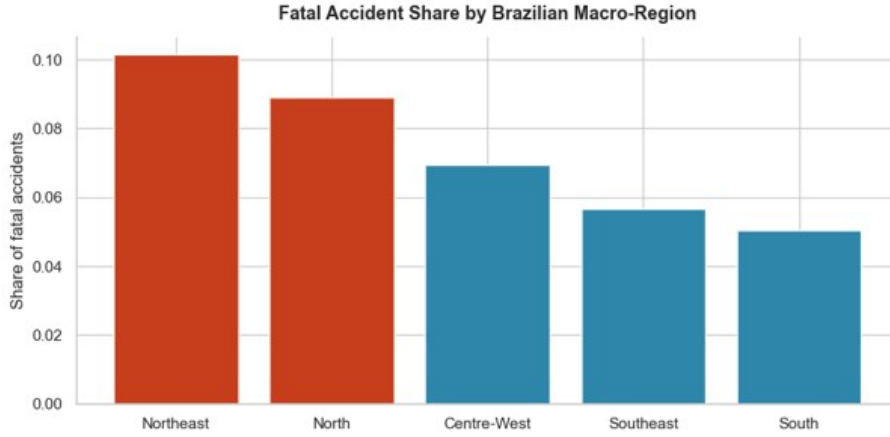


Figure 2: Fatal share by Brazilian macro-region.

An attribute that is potentially more specific than the region or state itself is Road_ID. The map created in Figure 3 shows where fatal accidents occur most frequently. A total of 7,917 fatal accidents occurred on highways BR-116 and BR-101. It is important to mention that these roads are among the longest in Brazil. It is understandable why their numbers are this high. [Ahmed et al. \(2023\)](#) says that road is one of the most important predictors of severity for New Zealand crash data. This gives an opportunity to assume the same applies to this dataset. Therefore, the feature is expected to be highly valuable for the SHAP analysis.

Table 7: Top ten Brazilian federal highways by fatal accident count

Federal highway	Accidents with dead victims
BR-116	4,097
BR-101	3,820
BR-153	1,394
BR-381	1,140
BR-040	1,099
BR-316	1,087
BR-364	1,029
BR-163	982
BR-230	842
BR-277	838

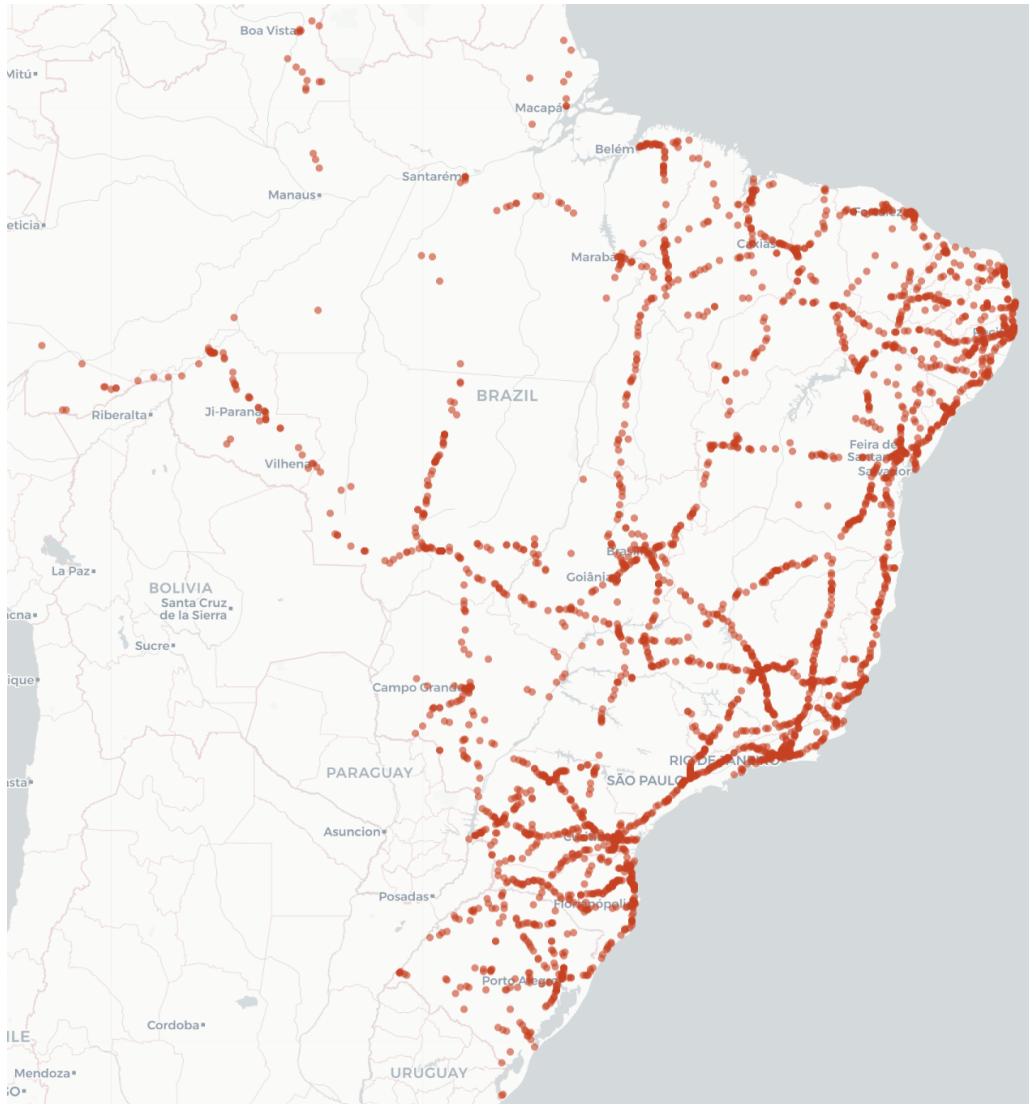


Figure 3: Spatial distribution of fatal accidents on Brazilian roads.

3.6.3 Temporal Patterns

The EDA of this theme found that there are different results when comparing weekdays and weekends, even though the days are evenly distributed. Furthermore, it was discovered that Weekends are more deadly than weekdays, especially during the night hours (22:00–05:00). What was also noticeable was that the results for Saturdays and Sundays differ, with Sundays being more deadly. This should be an adequate justification for retaining the Day_Type attribute, which has three categories (Weekday, Saturday, Sunday).

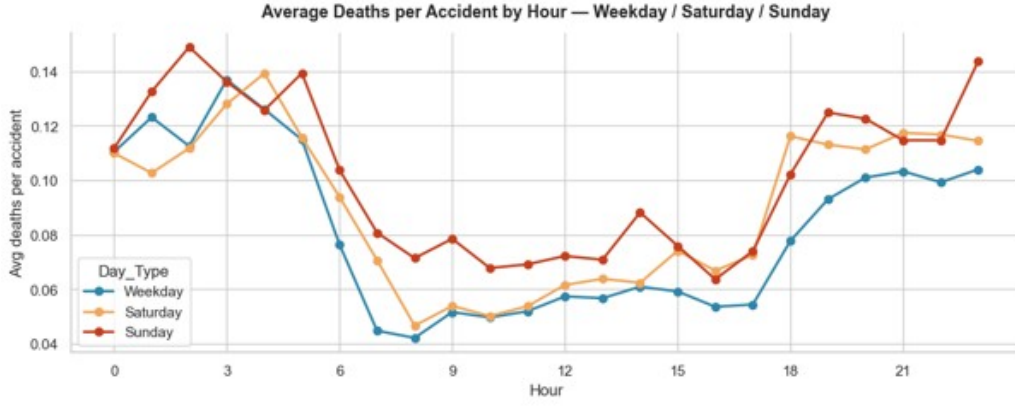


Figure 4: Hourly severity by Day_Type.

3.6.4 Accident Causes

Attribute Accident_Cause has over 80 categories. Most of them are similar and could be merged to create broader categories. For this purpose, 11 categories were created with the aim of developing a more robust and compact predictor. This suggests that half of all accidents are caused by drivers' behaviour on the road. However, it is interesting to see that these accidents are not the most frequent cause of death. The deadliest category is Pedestrian-related, which contains 30.9% of the total fatal share. Next is Driver state (fatigue/medical/distraction) and Visibility & signage, which over the overall fatal share as well. The surprising categories, which are below the index in Figure 5, are Alcohol/Drugs and Vehicle defect. It would be expected that these categories would rank higher, as they are commonly known for being massive contributors to traffic accidents. However, it is possible that safety measures are helpful enough that these categories are lower than expected. This does not contradict the findings of the study [Ahmed et al. \(2023\)](#), where alcohol and drugs were strong predictors for New Zealand crash data, because it secured third place in the table in terms of the number of accidents. This discovery also confirms the findings of the study [Scholz et al. \(2024\)](#), which states that accidents involving pedestrians have more serious consequences. Nevertheless, there is a high chance that Cause_Group will be an important predictor.

Table 8: Accident causes by volume, with fatal share for each category.

Cause_Group	Accidents	Share of all	Fatal share
Driver behaviour	227,662	49.2%	6.5%
Speed & following distance	75,854	16.4%	5.8%
Alcohol / Drugs	36,074	7.8%	5.4%
Vehicle defect	34,529	7.5%	2.9%
Road condition / infrastructure	28,601	6.2%	4.2%
Driver state (fatigue / medical / distraction)	21,614	4.7%	9.4%
Pedestrian-related	14,569	3.1%	30.9%
Animals	10,533	2.3%	5.0%
Visibility & signage	5,372	1.2%	8.4%
Weather	4,525	1.0%	4.7%
Other	3,800	0.8%	5.6%

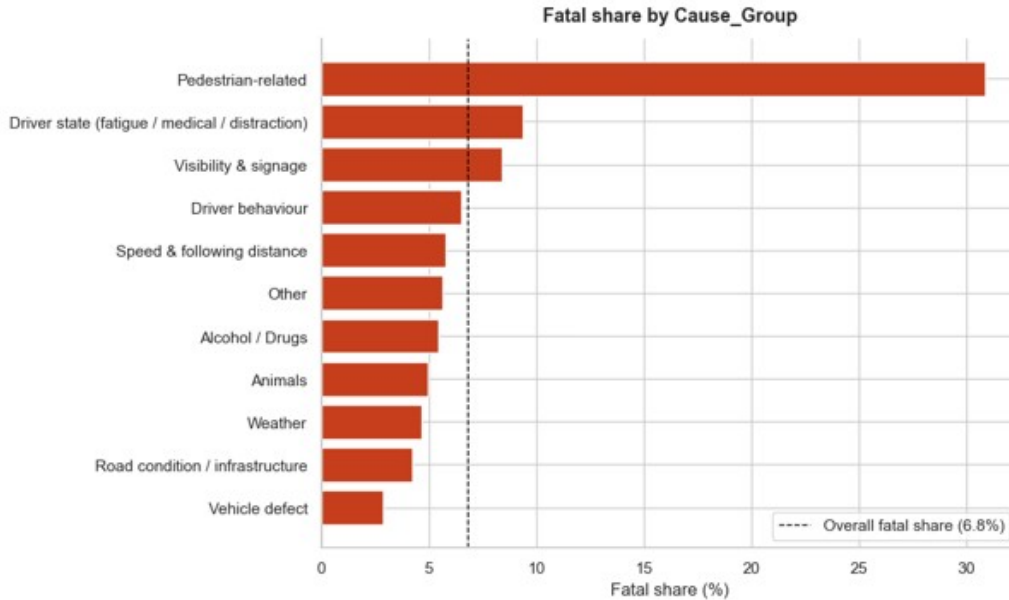


Figure 5: Fatal share by attribute Cause_Group.

3.7 Phase 1 — Data Preparation and Evaluation Module

Phase 1 focused on preparing data for further phases. The `random_state` was set to 25 for reproducibility for further research. The data were split in a ratio of 80 train to 20 test for target variable `Victims_Condition`. It was decided to save the split as parquet (file format) to preserve category, bool, and int32 data types on reload. The final train set contains approximately 370,506 rows and test set contains approximately 92,627 rows. Reusable evaluation function was created for this project to compute accuracy, macro-F1, per-class F1, MCC, and confusion matrix. The final outcome of this function is a DataFrame with a single row, which allows to display the number of these metrics side by side. The `DummyClassifier` model (official name from the library `scikit-learn`, which predicts the majority class and ignores input features) was used to establish a benchmark which other models must outperform. It achieved an accuracy of 0.7173, a macro-F1

score of 0.2785 and an MCC of 0.0000. It was determined that the main metric is macro-F1, and therefore every model must exceed a value of 0.28.

3.8 Phase 2 — Six-Classifer Baseline

Based on Section 2, it was decided to use six classifiers. In this phase, no imbalance-handling technique was applied. The six classifiers, which were compared, are:

- **Logistic Regression**
- **Decision Tree**
- **Random Forest**
- **XGBoost**
- **LightGBM**
- **CatBoost**

To achieve a fair comparison, a single one-hot encoding (OHE) pipeline was applied for all 6 models. `StandardScaler` was used on numeric categories and `OneHotEncoder` with `handle_unknown='ignore'` and `sparse_output=True` on categoricals. 5-fold stratified CV was used only on the training set to prevent potential leakage by implicitly "tuning" a specific test set and to obtain more stable scores by averaging over 5 different train/validation splits.

All six classifiers were able to achieve a higher macro-F1 score than the baseline. Direct comparisons with prior studies ([Scholz et al. 2024](#), [Gabardo et al. 2026](#)) are limited as binary classification was used for them. Even though the macro-F1 scores are lower than accuracy, this comparison is redundant as both metrics evaluate different values. Accuracy is prioritising the majority class, as it measures overall correctness. On the other hand, macro-F1 evaluates each class equally.

The `DecisionTree` model ranked first, even though the recent papers [Grinsztajn et al. \(2022\)](#), [Shwartz-Ziv & Armon \(2022\)](#) both argued gradient boosters should dominate tabular data. This contradicts their claims. This fact cannot be overlooked or left unaddressed, because if the selection of the three best models were incorrect, the following phases would carry this error forward and could lead to different conclusions. Therefore, a sanity check for the `DecisionTree` model was created to understand whether the dataset genuinely favours a single deep tree or if the model is over-fitted. The sanity check tested two diagnostics. The first diagnostic tried to fit `DecisionTree` on the full training set and predict on the sealed test set. The second diagnostic re-ran 5-fold CV with the tree depth capped at 20 instead of unrestricted. Both diagnostics led to the same conclusion. The `DecisionTree` is legitimate. A single tree captures a strong categorical signal from the dataset.

After verifying the validity of the `DecisionTree`, the following three best models were selected based on the macro-F1 score:

- **Decision Tree**

- CatBoost
- XGBoost

3.9 Phase 3 — Imbalance Handling Experiments

Three classifiers were selected based on their macro-F1 score and combined with theoretically seven imbalanced handling methods based on the literature review (Chawla et al. 2002, He et al. 2008, Tanha et al. 2020). This phase follows on directly from research question 2, and these are the seven imbalance-handling methods tested in Phase 3 were:

Table 9: The seven imbalance-handling methods tested in Phase 3.

Method	Approach
None	Control baseline
RandomOverSampler	Duplicate minority rows
RandomUnderSampler	Remove majority rows
SMOTE	Interpolate synthetic minority rows
ADASYN	Adaptive SMOTE for hard regions
SMOTE+Tomek	SMOTE + boundary cleanup
Class weighting	Weight minority errors more

For each sampler, the `sampling_strategy='auto'` was applied automatically to follow the established methodology proposed by Chawla et al. (2002) and minimise project complexity.

The critical methodological decision was to move from `sklearn.Pipeline` to `imblearn.Pipeline`. The problem is that SMOTE, ADASYN, etc. create synthetic samples by interpolation, which, if applied before cross-validation splitting, can get mixed with the validation folds, which would make models be partially evaluated on the interpolated synthetic samples. The solution to this problem would be for each sampler to be applied only to the training fold during each fold of CV and leave the validation untouched. This led to the use of the `imblearn.Pipeline` library, as this version of the library is capable of accurately resolving the issue and preventing any data leakage.

Only the combination of XGBoost and ClassWeight needed a manual CV loop. It is a combination where it needs to use `sample_weight`, which `cross_val_score` cannot pass. This led to a decision for this one experiment to create a manual 5-fold loop that computes `sample_weights` fresh for each training fold. This made it possible to proceed with 21 combinations and compare each of them fairly.

This led to a result where the top 3 combinations (CatBoost + RandomOver, XGBoost + RandomOver, XGBoost + ClassWeight) were selected, saved as a Joblib file and carried forward to Phase 4.

3.10 Phase 4 — Hyperparameter Tuning

Optuna with Bayesian optimisation was used to find the best possible hyperparameters. It is a library that tries to find the best combination automatically, and with Bayesian optimisation, it can learn from each trial until it gradually achieves the best possible result. Seed number 25

was selected on the grounds of reproducibility, in case a study were to emerge in the future that wished to expand this project.

Each combination has different possible hyperparameters; therefore, three separate studies were carried out in order to obtain clear results. Fifty trials were conducted in each study; this amounted to a total of 150 experiments, which took approximately 10 hours.

5-fold stratified CV with the same random state 25 was performed in each Optuna trial. This should ensure a fair comparison across all phases.

The search range numbers were selected based on the official default documentation of both models, with prior tuning work in the tabular machine learning literature.

Table 10: Hyperparameter search spaces used in Phase 4 Optuna studies.

Classifier	Hyperparameter	Range	Scale
XGBoost	n_estimators	100–800	linear
XGBoost	max_depth	3–10	linear
XGBoost	learning_rate	0.01–0.3	log
XGBoost	subsample	0.5–1.0	linear
XGBoost	colsample_bytree	0.5–1.0	linear
XGBoost	min_child_weight	1–10	linear
XGBoost	reg_alpha	0–1	linear
XGBoost	reg_lambda	0–1	linear
CatBoost	iterations	100–800	linear
CatBoost	depth	4–10	linear
CatBoost	learning_rate	0.01–0.3	log
CatBoost	l2_leaf_reg	1–10	linear
CatBoost	bagging_temperature	0–1	linear

While studies were running, a method was approached to insert a safety net. Optuna enables an option to recover from a potential crash by storing the trial results in a SQLite database file.

The best combination proved to be XGBoost + RandomOverSampler, which achieved a CV macro-F1 score of 0.5188. In the previous phase, the CatBoost + RandomOver combination (0.4825) proved to be the most successful, which contributed to the decision to carry out phase 4, as it allows us to assess the significance of hyperparameter tuning, rather than continuing with the base combination.

A possible methodological approach could be to first perform hyperparameter tuning and then imbalance handling experiments. This variant is further discussed in Section 5.

3.11 Phase 5 — TreeSHAP Explainability

TreeSHAP was selected for Phase 5, which is a specific variant of the SHAP algorithm that is optimised for tree-based models, to explain the value of each feature. Other options such as LIME would provide only local feature ranking, not global. Furthermore, the study of [Lundberg & Lee \(2017\)](#) demonstrated that TreeSHAP satisfies all three important properties, namely: Local Accuracy (the sum of contributions exactly matches the model’s output), Consistency (the importance of a feature is never reduced when its actual contribution increases) and Missingness

(unused features have a zero contribution). Based on these findings, TreeSHAP was the best option for feature explainability in this project.

It was decided to perform two computations, which were compared in order to verify the reliability of the SHAP method's results. The first computation was conducted on a small subset of the test dataset (10,000 rows), which was selected to maintain class balance. The result was obtained within one minute and was also used for visualisation. The second computation was intended to verify the reliability of the first computation using the entire test set of 92,617 rows. The two computations agreed within 1%. This confirmed the reliability of the first computation and validated its results.

The 398 binary columns using the one-hot encoding method had to be converted back to the original 22 features, as otherwise TreeSHAP would have generated values for all 398 columns. As well as, to ensure that the results were readable and interpretable, it was therefore decided to convert them back. For example, the five 'Region_BR' columns with OHE encoding were merged into a single 'Region_BR' column expressing importance.

Nine different charts were generated across five categories. An overall bar chart was created to illustrate the average ranking of features across all classes. A 'beeswarm' chart was created to visualise the distribution of SHAP values by individual features. To understand which feature influences the severity of each class, a bar chart was created for each class. For the feature with the greatest influence on the class containing fatal cases, three dependency plots were created, showing how the SHAP value changes as the feature's value changes. Finally, a single waterfall plot was generated, which was able to predict one fatal case and which illustrates how each feature shifted the model's prediction from its initial value to the final output.

All SHAP values were saved as compressed files in .npz format so that they could be reused if necessary.

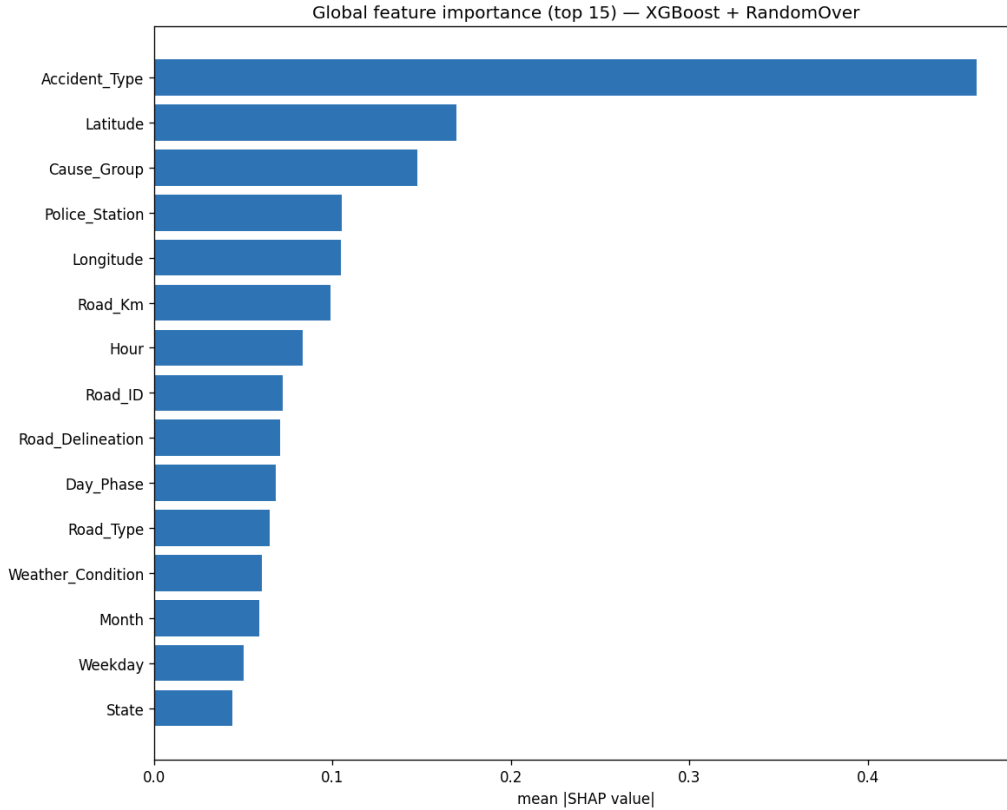


Figure 6: Example of a global bar plot generated by TreeSHAP, showing the top 15 features ranked by mean absolute SHAP value.

3.12 Metrics and Evaluation

The macro-F1 score was chosen as the primary metric for this project. Unlike accuracy, which is not suitable for imbalanced data and would select a predictor with a larger proportion of data. A clear example is `DummyClassifier`, which achieved an accuracy of $\tilde{71.7\%}$, but only achieved a macro-F1 score of $\tilde{27.9\%}$. Macro-F1 calculates the F1 score separately for each class and, when averaging the scores, ignores class density. Consequently, the majority class is treated in the same way as the minority class, and vice versa.

The MCC (Matthews Correlation Coefficient) metric was chosen as a secondary metric, based on the study [Tanha et al. \(2020\)](#). The interpretation of the MCC metric is straightforward, as its values range from -1 (completely incorrect) through 0 (no predictive ability, corresponding to a random predictor or a predictor based on the majority class) to 1 (perfect prediction). Another example of the `DummyClassifier` model, which achieved an MCC value of 0.0000, meaning that the model failed to learn anything.

In addition to the macro-F1, the per-class F1 was also used to assess the model’s performance for each severity class in greater detail, as it is important to have an overview of the performance of each class, and not just the performance of the model itself, especially for the fatal class.

Finally, in order to obtain a proper, detailed analysis, a confusion matrix was chosen, as it reveals the model’s errors by highlighting both the model’s correct predictions and the instances where it was incorrect.

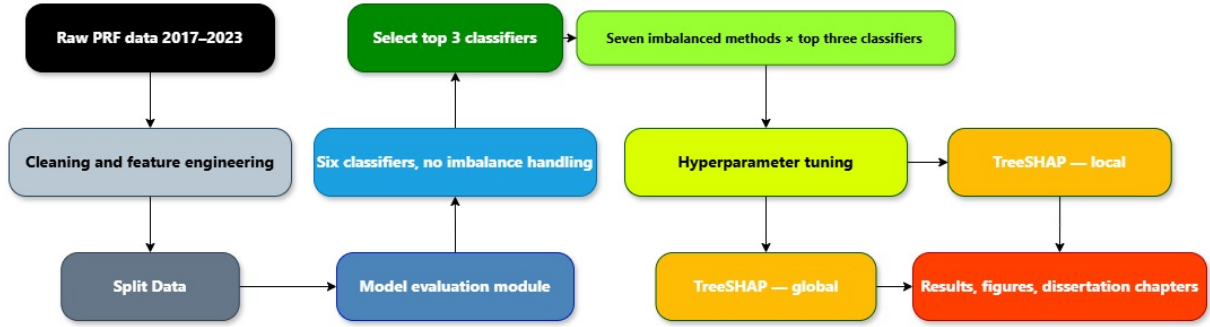


Figure 7: Overall artefact development pipeline

4 Results and Discussion

4.1 Phase 2 Baseline Results

Table 11 shows the results of each model from Phase 2. A well-received result was that all models were able to outperform the baseline DummyClassifier and achieved a score higher than 0.2785. This implies that all models were able to learn genuine patterns. The result from this Phase also preliminarily answers research question 1. Almost all tree-based classifiers (DecisionTree, CatBoost, XGBoost, and RandomForest) were able to outperform Logistic Regression. The only model which was not able to outperform Logistic Regression was LightGBM. The difference is not very significant (LogisticRegression = 0.4043, LightGBM = 0.3985); therefore, the difference is possibly caused by the default settings of both models.

The results obtained from this iteration would, with high probability, be the same if it were repeated, as the standard deviation is ≤ 0.003 ; the ranking would therefore remain unchanged, which confirms the selection of the three best models for Phase 3.

As already mentioned, DecisionTree surprisingly proved to be the best-performing model, achieving a macro-F1 score of 0.4353. Studies in the field of machine learning, such as [Grinsztajn et al. \(2022\)](#), [Shwartz-Ziv & Armon \(2022\)](#), state that all gradient boosters (CatBoost, XGBoost, LightGBM) should dominate when dealing with tabular data. Given that the DecisionTree model performed best, this contradicts their assumptions. For this reason, a sanity check was conducted to confirm the result.

Two diagnostics were performed to confirm the model’s credibility. The first diagnostic trained on the full training set and predicted on the sealed test set. The result of CV macro-F1 for this diagnostic was 0.4397, which was a little bit higher than the previous score (0.4353). If it were lower, it would mean that the DecisionTree was overfitting the CV folds. The second diagnostic was to set up the ‘max_depth’ to be 20 and re-run the 5-fold CV. This result to be 0.4280 of macro-F1, which is almost identical to the unrestricted version. If it were lower, it would mean that the tree was memorising and was not able to capture the signal at shallow depths. These diagnostic tests excluded the possibility that the model was over-fitted and therefore confirm that the model is reliable and that the result should be considered valid.

The only explanation for why DecisionTree emerged as the best model is the data set itself. It is possible that the categorical transformed variables (such as Accident_Type and Cause_Group) contain such a strong signal that even a simple tree is able to capture it. This

is a specific feature of this particular data set, which can be explained further in SHAP Feature Importance Findings.

This phase revealed certain limitations, such as the fact that the default LightGBM settings achieved lower results than expected, and that a potentially fine-tuned LightGBM might outperform the other models. However, this is merely a conjecture that has not been verified.

The last thing which should be mentioned is the time it took RandomForest (1185 seconds) to complete 5-fold CV. It took approximately 85–90× longer than XGBoost while still achieving a lower score than XGBoost. This shows great potential for gradient boosting models as practical evidence for tabular data.

Table 11: Phase 2 baseline results: six classifiers ranked by 5-fold stratified CV macro-F1 (no imbalance handling).

Rank	Classifier	CV macro-F1	Std
1	DecisionTree	0.4353	0.0016
2	CatBoost	0.4188	0.0029
3	XGBoost	0.4159	0.0029
4	RandomForest	0.4063	0.0017
5	LogisticRegression	0.4043	0.0016
6	LightGBM	0.3985	0.0020
—	DummyClassifier (majority)	0.2785	—

4.2 Phase 3 Imbalance-Handling Results

Table 12 shows the best combination out of all 21 variants. Two patterns emerged on the surface. The best classifiers appeared to be XGBoost and CatBoost. The best imbalance-handling technique appeared to be RandomOver and ClassWeight. Furthermore, all combinations sit within 0.003 macro-F1 of each other CatBoost + RandomOversampler (0.4825), XGBoost + RandomOversampler (0.4811), XGBoost + ClassWeight (0.4804), and the fourth CatBoost + ClassWeight (0.4799). Overall, the result was unexpected, as techniques such as SMOTE – which is typically recommended for imbalanced data – achieved less favourable results. The same applies to ADASYN.

The SMOTE method achieved poorer results in these calculations, specifically because of high-cardinality one-hot encoded categorical features such as Road_ID or Police_Station and the generation of synthetic samples. The former allows the dataset to be broken down into hundreds of sparse binary columns, whilst the latter produces synthetic results that cannot be applied to real-world traffic accidents. For example, SMOTE can create synthetic samples where Region_BR_Northeast = 0.6 and Region_BR_Southeast = 0.4, which is not possible, as it must take place in one specific region. This example can be applied to other columns such as Road_ID or Police_Station. The combination of these two properties resulted in a mismatch of the possible results SMOTE could offer. Whereas, on the other hand, the RandomOversampling method duplicates the actual minority rows. It avoids the problem with synthetic samples.

SMOTE was able to achieve the best macro-F1 combination around 0.46, ADASYN around 0.45, and SMOTE+Tomek around 0.46. The difference between the top combinations and these

is around 0.02 macro-F1. These results do not challenge the findings of the studies [Chawla et al. \(2002\)](#) and [He et al. \(2008\)](#). It merely highlights a possible limitation that these papers did not emphasise.

The interesting factor was the result of DecisionTree. In the last phase, DecisionTree was able to rank first among the others. In this phase, however, it was unable to benefit from any of the seven imbalance handling methods. The best result was obtained from the combination with nothing and scored 0.4358, essentially matching its Phase 2 result of 0.4353. This score was unable to beat any other combination, which had a similar result or scored lower. The worst result achieved was 0.3838 when using a combination with RandomUnderSampler. Thanks to the leaf nodes, which were not limited by depth, it was able to memorise the pattern for the minority class sufficiently, which was enough to reach the maximal potential from the beginning. In contrast, CatBoost was able to improve from 0.4184 to 0.4825 with the combination of RandomOver. It was able to gain 0.064 from the previous version. This supports the narrative that gradient boosters are based on loss function gradients and contain shallow trees, which enables them to respond better to imbalance handling methods.

Last noticeable results were combinations with SMOTE-Tomek. The three experiments took approximately 9 hours each of them, which makes around 27 hours in total. The result was not favourable, as it was not better than the standard SMOTE. This can be interpreted as SMOTE-Tomek was not ideal for this dataset, given its large volume, and that it is more suitable for use on smaller datasets.

Finally, this Phase was able to answer research question 2. It is visible that imbalance handling benefits booster models (CatBoost and XGBoost gained $\tilde{0.06}$ macro-F1), and simpler methods(RandomOversampler and native class weighting) are more favourable than the more complex ones (SMOTE, ADASYN, SMOTE+Tomek). That being said, this answer can only be applied to this dataset and similar ones, such as datasets with high-cardinality one-hot-encoded categorical features.

Table 12: All 21 combinations from Phase 3, ranked by 5-fold stratified CV macro-F1. The top 3 (in bold) were carried forward to Phase 4 hyperparameter tuning.

Rank	Combination	CV macro-F1	Std	Time (s)
1	CatBoost + RandomOver	0.4825	0.0021	205.5
2	XGBoost + RandomOver	0.4811	0.0022	25.4
3	XGBoost + ClassWeight	0.4804	0.0019	19.0
4	CatBoost + ClassWeight	0.4799	0.0021	101.5
5	CatBoost + RandomUnder	0.4697	0.0026	26.5
6	XGBoost + RandomUnder	0.4687	0.0029	7.3
7	CatBoost + SMOTE	0.4588	0.0027	933.0
8	CatBoost + SMOTE+Tomek	0.4581	0.0032	31,492.2
9	CatBoost + ADASYN	0.4521	0.0029	2,247.4
10	DecisionTree + none	0.4358	0.0017	212.2
11	DecisionTree + SMOTE+Tomek	0.4354	0.0020	32,051.2
12	DecisionTree + ADASYN	0.4353	0.0033	2,615.5
13	DecisionTree + SMOTE	0.4339	0.0028	1,112.6
14	DecisionTree + ClassWeight	0.4333	0.0012	176.6
15	XGBoost + SMOTE+Tomek	0.4326	0.0026	32,823.8
16	DecisionTree + RandomOver	0.4316	0.0023	460.4
17	XGBoost + SMOTE	0.4310	0.0027	208.4
18	XGBoost + ADASYN	0.4206	0.0021	1,484.6
19	CatBoost + none	0.4184	0.0023	96.7
20	XGBoost + none	0.4156	0.0021	20.4
21	DecisionTree + RandomUnder	0.3838	0.0015	17.7

4.3 Phase 4 Hyperparameter Tuning Results

The table 13 shows the results of Phase 4. It shows the best trial from each of the three Optuna studies. The winner is a combination XGBoost + RandomOver at trial 43 with CV macro-F1 0.5188. Right behind it is the combination XGBoost + ClassWeight, which performed the best at trial 49 and gained CV macro-F1 0.5181. The third and last place took the combination CatBoost + RandomOver, which gained from the last score (CV macro-F1 0.4825) by 0.019. The average gain is approximately 0.03 macro-F1, with the XGBoost combinations managing to take the top spot in the rankings and overtake the CatBoost + RandomOver combination, which held the top position from the third phase. This result shows that the number of trials was sufficient. It is possible that with just 30 trials, the result would not differ significantly, and the experiment would have ended after the last trial.

The winner (XGBoost + RandomOver) took about 3 minutes, whilst it took last place (CatBoost + RandomOver) about 13 minutes, which makes the winner 4 times faster per trial than last place during the tuning phase. Only about 2 minutes took XGBoost + ClassWeight, which makes it finish the tuning the fastest way, and it was only 0.0007 macro-F1 score behind the first place.

Table 13: Phase 4 hyperparameter tuning results. Best trial from each of the three Optuna studies, ranked by CV macro-F1.

Combination	Best trial	CV macro-F1	Phase 3 score	Gain
XGBoost + RandomOver	43	0.5188	0.4811	+0.038
XGBoost + ClassWeight	49	0.5181	0.4804	+0.038
CatBoost + RandomOver	48	0.5016	0.4825	+0.019

From each tuning result, two plots were created. A history plot showing how the model was evolving during each trial, and a parameter importance plot (bar chart) explaining which parameter affected the score most. For each history plot, there is a visible improvement from trial 1 to trial 10. It then gradually improves until trial 30, where the line of improvement subsequently flattens out completely. This shows a nice Bayesian optimisation pattern and justifies the selection of 50 trials, which is sufficient but not excessive for this study. The importance plot shows for XGBoost + RandomOver that the most important is the `max_depth` (importance = 0.59). The following is `learning_rate` (0.23). These two hyperparameters combined already reached 82%, which means Optuna was mainly focusing on them. This should be a takeaway for future studies, where the main focus should be on those two hyperparameters rather than the others (`reg_lambda`, `subsample`, `n_estimators`, `reg_alpha`, `colsample_bytree`, `min_child_weight`).

The limitation of this study is that the hyperparameter `max_depth` saturated at its upper bound across all three studies. The parameter was at the maximum of 10. This suggests that for future studies, the limit could be increased to 15 or 20 and the results could be compared, in order to increase the potential of the model and the precision of its predictions. The other hyperparameters were selected correctly and do not require any adjustments.

Overall, this phase was able to explore how hyperparameter tuning can change the final result. The best approach based on macro-F1 is the combination XGBoost + RandomOver (0.5188). The average gain (0.03) is appropriate and doesn't indicate any overfitting to the CV folds. The number of trials (50) for each combination is adequate and fair. The plots were created for better understanding of each trial and which parameter is the most important for each combination.

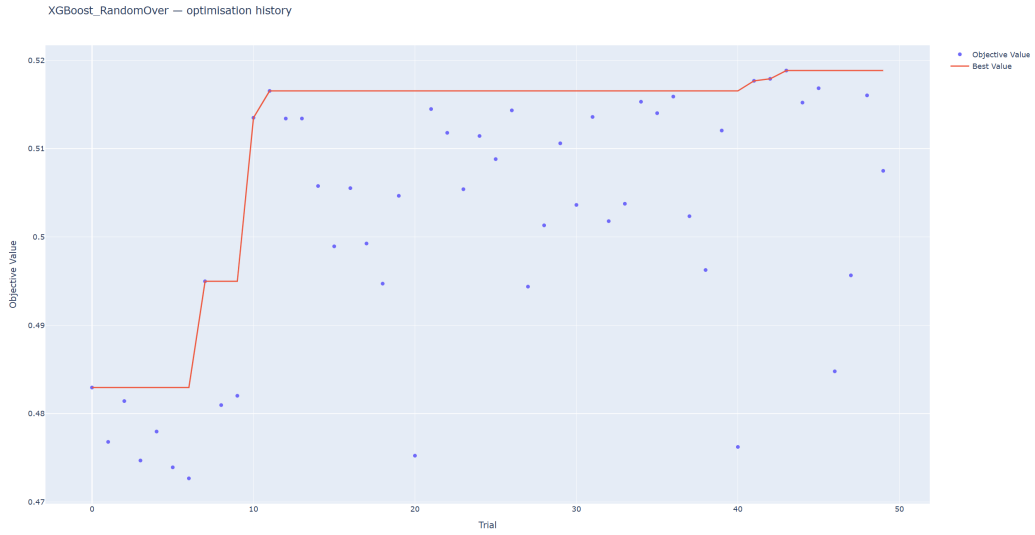


Figure 8: Optuna optimisation history plot of XGBoost + RandomOver (0.5188)

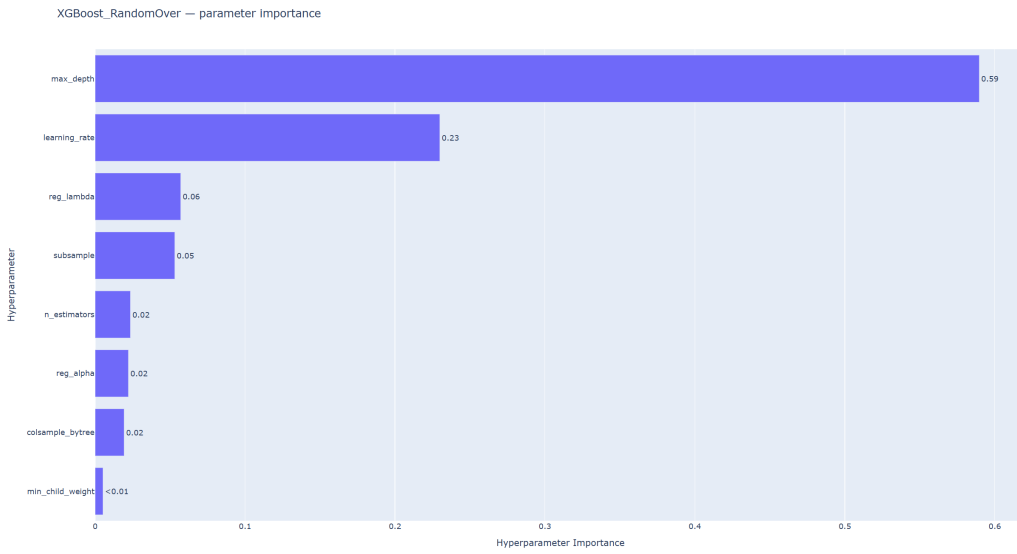


Figure 9: Optuna parameter importance plot of XGBoost + RandomOver (0.5188)

4.4 Final Test-Set Evaluation

The final metric results for the winning combination (XGBoost + RandomOver) are:

- Macro-F1 = 0.5194
- MCC = 0.2784
- Accuracy = 0.6306

This shows clear improvement when comparing to the DummyClassifier baseline (accuracy = 0.7173, macro-F1 = 0.2785, MCC = 0.0000). Furthermore, per-class F1 was important to assess in order to determine the probability of occurrence for each class. The per-class F1 are:

- With injured victims F1 = 0.7336

- Without victims F1 = 0.4638
- With dead victims F1 = 0.3607

It is not surprising that the class 'With dead victims' got the lowest result, as it was established before that it is a minority class. However, the final combination improved over the DecisionTree baseline from Phase 2 (from F1 = 0.2166 to 0.3607). It is clear that the model has managed to learn to identify patterns, which has enabled this minority group to achieve a higher score than before. Another fact is that the CV folds were not overfitted and there is no visible bias. The model is capable of performing well on unseen data, as the CV macro-F1 from Phase 4 was 0.5188 and the test set macro-F1 is 0.5194. The difference is minimal (difference of 0.0006, well within CV noise). The tuning process not only enhanced the CV performance but also the best-performing model configuration.

The final recall is more than doubled if we compare it to the sanity check from Phase 2. The DecisionTree was able to achieve 22.4% of fatal cases on the test set (1,403 of 6,269) and the final tuned XGBoost + RandomOversampler model correctly identifies 50.1% (3,141 of 6,269). Even though the class 'With dead victims' F1 is only 0.3607, the final results show the possible improvements with the systematic imbalance-handling and hyperparameter tuning approach. All of this could be achieved without any catastrophic loss of precision.

Table 14 shows the full confusion matrix for the final evaluation. It shows a credible source of information and supports the conclusion in the last paragraph. The table also reveals a trade-off, which is visible for class 'With injured victims'. The misclassification of 6,973 injured victims as fatal shows the consequences of the imbalanced handling. The RandomOversampling method is pushing the narrative of the minority class to be given more weight while training. That being said, it is more acceptable to have a model which will predict a false-fatal case than a missed-fatal case. Therefore, such a trade-off should be acceptable and acknowledged, especially when it comes to confidentiality and the safety objective of this thesis.

Table 14: Confusion matrix for the final tuned XGBoost + RandomOversampler on the sealed test set.

True \ Predicted	With dead	With injured	Without
With dead victims	3,141	2,229	899
With injured victims	6,973	44,450	15,013
Without victims	1,034	8,063	10,815

As visible in Table 15, there is a clear difference between Macro-F1 and MCC. The primary metric, macro-F1, increased from 0.2785 to 0.5194, and MCC increased from 0.0000 to 0.2784. The accuracy, however, decreased from 0.7173 to 0.6306. The reason for this is the attempt to correctly predict all three classes, unlike what the DummyClassifier demonstrates. Therefore, accuracy decreased because the combination is correctly trying to predict accident severity, which is a positive trade-off when it comes to road safety applications.

This is the final result of the six-phase methodology, which would be impossible to achieve with a naive prediction strategy. There are no other studies with which this result could be compared. This is the first study to perform a three-class benchmark on the national PRF

dataset with imbalance handling techniques and proper hyperparameter tuning. The previous studies [Scholz et al. \(2024\)](#) and [Gabardo et al. \(2026\)](#) were only focusing on binary classification. Therefore, these results can be taken for further studies as a stepping stone.

Table 15: Final test-set performance of the tuned XGBoost + RandomOversampler compared to the DummyClassifier baseline.

Metric	DummyClassifier	Final tuned model
Accuracy	0.7173	0.6306
Macro-F1	0.2785	0.5194
MCC	0.0000	0.2784
F1 [With dead victims]	0.0000	0.3607
F1 [With injured victims]	0.8352	0.7336
F1 [Without victims]	0.0000	0.4638

4.5 SHAP Feature Importance Findings

Table 16 shows the final global result of TreeSHAP analysis. It is represented by the mean absolute SHAP value of the top 10 features. This value indicates the extent to which a given feature will influence the prediction of the type of accident. A feature 'Accident_Type' with a SHAP value of 0.453 dominates the ranking. It is three times larger than the second feature, 'Latitude' (0.169). This is the strongest predictor in the global explanation and can be interpreted as the main source of the accident. This should suggest safety improvements in categories such as physical collision dynamics ('Rear-end collision', 'Collision with fixed object', 'Overturn', Etc.) rather than focusing on time, location, or weather. To verify the reliability of global SHAP results, a second computation was made, which mainly focused on the full 92,617-row test set. The key takeaway was that the top ten feature rankings agreed within 1%. This is solid evidence of the first computation being reliable and valid.

Table 16: Top 10 features ranked by mean absolute SHAP value across all three severity classes, computed on the full 92,617-row test set.

Rank	Feature	Mean SHAP
1	Accident_Type	0.453
2	Latitude	0.169
3	Cause_Group	0.149
4	Police_Station	0.106
5	Longitude	0.104
6	Road_Km	0.099
7	Hour	0.083
8	Road_ID	0.072
9	Road_Delineation	0.070
10	Day_Phase	0.068

As visible in Table 17, there are noticeable differences between the rankings for the top-five features per severity class. The first place still takes Accident_Type, but the differences are gradually becoming more significant. The last row indicates which features are more important for predicting a particular class.

This interpretation is based on findings from EDA and SHAP analysis. The EDA mentioned in Figure 4 a higher death rate during the night. This corresponds with the results of the ranking of the five most significant features in each class. The reason is the Day_Phase column, ranked amongst the top five features for 'With dead victims'. For the other classes is a different story, where the column can be found, for example, at 14th place for 'With injured victims'. It is therefore possible, based on the time of the day, to predict whether an accident becomes fatal or injurious.

An interesting second per-class finding is that 'Weather_Condition' was able to rank fifth for 'Without victims' predictions but could not reach the top 10 for other classes. This can be interpreted as weather like rain, fog, or poor visibility does not cause severe accidents. Furthermore, it can be interpreted as during severe weather, people on the roads take greater care, which in turn leads to fewer serious accidents. This finding was unexpected, as one might have expected the opposite result. But the results speak for themselves, and that must be respected despite all the doubts.

The column 'Latitude' was able to reach the top 3 in each class and global ranking. Therefore, a dependence plot was generated to interpret and understand the result. In Brazil, latitude can range from approximately -33 (southernmost) to +5 (northernmost). StandardScaler was used to subtract the mean and divide by the standard deviation. Therefore, the plot should show, from left to right, from South to North. It shows how fatality rises based on the position. Southern regions show a lack of fatality (SHAP values reaching around -1.3), whereas, on the other hand, Northern regions are likely getting a positive outcome (SHAP values reaching around +0.5). This indicates that the model was able to learn and recognise a pattern, which can be interpreted as the North contains a strong fatal signal and the South contains a strong non-fatal signal. A similar story was already shown in EDA (Figure 2), where the Northeast region has twice the fatal share (10.2%) than the South region (5.1%). These two conclusions support one another, particularly as they were unaware of each other's existence.

Table 17: Top 5 features per severity class, ranked by mean absolute SHAP value within each class.

Rank	With dead victims	With injured victims	Without victims
1	Accident_Type	Accident_Type	Accident_Type
2	Latitude	Latitude	Cause_Group
3	Cause_Group	Cause_Group	Latitude
4	Day_Phase	Longitude	Police_Station
5	Road_Km	Police_Station	Weather_Condition

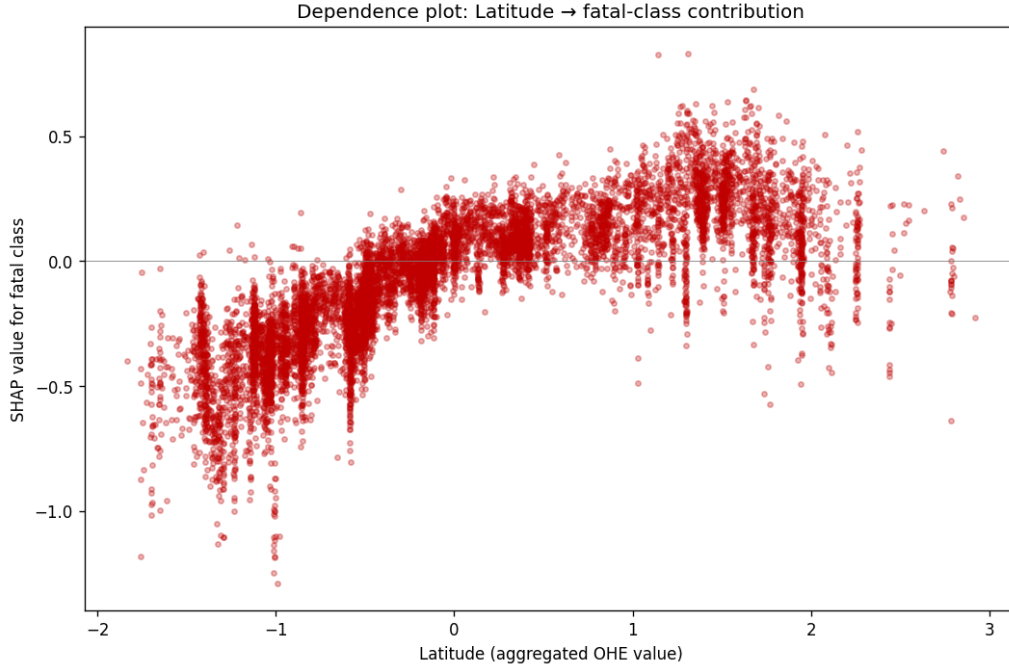


Figure 10: Dependence plot of Latitude for global ranking

A single waterfall plot, which can be seen in Figure 11, was created to illustrate and see the model behaviour. It shows one correctly predicted fatal case for the test set. At the start, the model is assuming, without any additional features, that the accident will not be fatal. This can be seen in the baseline, which contains $E[f(X)] = -0.129$. The most important features pushed the model to the narrative that the accident is fatal, as `Accident_Type_Head-on collision` = +1, `Latitude` = +0.2 and `Cause_Group_Driver` state (fatigue/ medical/ distraction) = +0.1, even though the accident happened on Thursday, which only gives -0.11. From the baseline of -0.129 to the final value of +1.43, each feature contributed to the final result. This also shows how a strong signal can create the `Accident_Type` feature. To interpret the plot in plain language. The model was able to assume that the collision is fatal based on a head-on collision, the location(North) and possible driver fatigue despite the accident happening on Thursday.

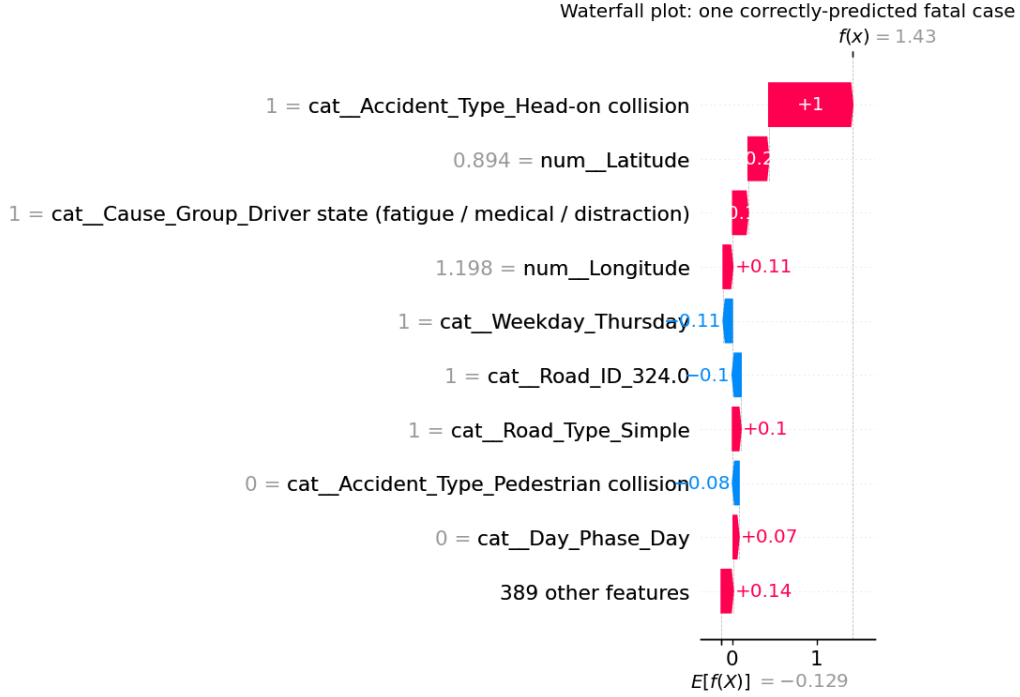


Figure 11: Waterfall plot to determine the type of the accident

An important fact to mention is that the feature `Police_Station` reached 4th position in the global ranking. Police stations or police officers are not the cause of the traffic accident. It is more like a regional or geographic proxy. Each police station covers a distinct area with different road quality, traffic density, highway characteristics or reporting practices. Therefore, a `Police_Station` serves as a label for geographic/infrastructural factors that actually drive severity. Building a new police station is not a solution to reduce the number of accidents.

The final phase was able to answer the last research question. For the global ranking, the ‘`Accident_Type`’ is the most important feature in terms of traffic accident severity. The following features are `Latitude`, `Cause_Group`, `Longitude` and `Road_Km`, which contain geographical and cause-based attributes. Each class has a gradually different ranking. As for fatal accidents, the important features `Day_Phase` and `Road_Km` ranked in the top 5. `Weather_Condition` was able to rank only in class ‘Without victims’, which was interpreted as drivers being more careful during the weather.

5 Conclusions and Future Work

5.1 Summary of Contributions

This final thesis filled gaps established in the Literature Review (Section 2).

The previous studies([Scholz et al. \(2024\)](#), [Gabardo et al. \(2026\)](#)) address only binary classification for the PRF dataset. This is the first study to address severity on the full national PRF dataset as a problem for multidimensional classification and has succeeded in providing qualitative insights and results.

Furthermore, it was able to perform a comparison of 7 imbalance handling methods across three tree-based classifiers in a fair, cross-validated framework. This is therefore the first study

in which several techniques for addressing imbalance have been applied to the PRF national dataset. Previous studies either ignore the problem or implement only one technique, SMOTE.

The third gap was filled by implementing modern explainability at the country level for PRF severity prediction. The first study, which was able to use the TreeSHAP algorithm and analyse the result by creating a global ranking, per-class comparisons, dependence plots, and a waterfall plot. This should be able to provide stakeholders or policymakers with necessary information regarding road safety and the severity of the situation.

Besides the gaps, which were established before, this work was able to discover interesting insight. It was discovered that a simple resampling technique can outperform sophisticated SMOTE on OHE high-cardinality data. This can be interpreted as a new finding for future studies that seek to investigate a similar problem.

The final tuned model was able to reach more than double the fatal-class recall. This indicates a significant improvement, as the DecisionTree baseline from Phase 2 achieved 22.4% of fatal cases (1,403 of 6,269). However, the final tuned model emerged to achieve 50.1% (3,141 of 6,269). This demonstrates the importance and necessity of all the phases that have taken place.

5.2 Answers to Research Questions

This thesis was able to answer all 3 research questions. The answers are supported with well-written reasons with meaningful insights.

- **RQ1 - Tree-based Classifiers vs Logistic Regression:**

This research question was possible to answer in Phase 2. The final answer for this question is yes with one exception. The tree-based classifiers (DecisionTree, CatBoost, XGBoost, and RandomForest) were able to beat Logistic Regression except for LightGBM. Logistic Regression was able to score 0.4043, and the best tree-based classifier in this phase was able to score 0.4353. Furthermore, in Phase 5 and after tuning, the best model (XGBoost) and its imbalance handling technique (RandomOver) did achieve 0.5194.

- **RQ2 - The Effect of Imbalance Handling and a Comparison:**

The evidence to answer this research question was possible to get through the systematic Phase 3 imbalance experiments. The imbalance handling techniques have succeeded in increasing the macro-F1 score of their models, except for DecisionTree, which did not benefit from any of the seven methods. CatBoost with RandomOver gained 0.064 and became the best-performing model in Phase 3. A new finding was also discovered that simple methods such as RandomOver and ClassWeight are a better option for this specific dataset than the complex ones.

- **RQ3 - Feature Importance:**

Globally, the most important feature is Accident_Type, which dominates $\sim 3\times$ more than the second most important feature, Latitude. Next in line was Cause_Group, and that was followed by geographic features (Police_Station, Longitude, Road_Km). Per-class analysis

revealed Day_Phase is important for fatal accidents. It was able to rank the forth place, but fourteenth for the injured class. On the other hand, the Weather_Condition column was only ranked in the top 5 features for the class 'Without victims'. This indicates the weather being responsible for minor non-lethal collisions but not for injured or fatal cases.

5.3 Limitations

1. The first limitation of this study is not selecting LightGBM for tuning Phase. LightGBM performed the worst out of all six classifiers for this dataset. That being said, its default setting could cause the result. Therefore, this does not reflect its potential true capabilities. This could act as a stepping stone for future research or experiments, potentially leading to a completely different outcome.
2. The second limitation could lie in Hyperparameter tuning ranges. Almost all hyperparameters were able to fit within their ranges. Only max_depth/depth reached its maximum potential (max_depth/depth = 10). Therefore for future studies, the implementation of wider ranges for this parameter could lead for further performance improvements.
3. Another limitation is caused by Police_Station, which came fourth in the global SHAP ranking. The Police_Station does not cause the traffic accident. This has more to do with regional areas rather than causal effect. Police makers and stakeholders must be well informed about this limitation to be able to properly reduce accident severity.
4. A possible variant of SMOTE should be recommended for future categorical datasets. The basic variant of SMOTE was designed to handle continuous numeric features rather than sparse binary columns. Therefore, implementing SMOTE-NC, which is a special variant that can handle categorical and numerical features, should be appropriate to use in future studies. This imbalance technique must be applied before the OHE is implemented in order to realise its full potential.
5. In this study, the imbalance handling techniques were performed before the tuning phase. An alternative approach could be to first tune the top 3 models and then perform the imbalance experiments. This could bring new findings such as the top 3 rankings from Phase 2 could change and tuned classifiers would have different responses from the default ones. This would also show a comparison of the final results, where it would be possible to assess if such a methodological change impacts the final performance.
6. The last limitation was caused by interacting with the test set. There were two instances where the test set was used and touched. The first was a sanity check for DecisionTree, which was made to identify the possibility of an overfitted model. The second time was in Phase 5, which was an intended step for final assessment. The planned interaction with the test set was only applied for Phase 5. That being said, this approach did not influence the final results, but should be mentioned as a limitation for potential unexpected factors that may arise during the experiments.

5.4 Future Work

1. Group 1: Methodology extensions

Future studies could extend the methodology by implementing SMOTE-NC instead of standard SMOTE. Using SMOTE-NC effectively would require running on the raw categorical features before OHE, which would change the approach that must be taken into account. Another possible extension would be to tune LightGBM to discover whether tuning can change the ranking at Phase 2. The final extension would be to change the range of max_depth/depth upper bound from 10 to 15. This means the model can get the deeper range which was required.

2. Group 2: Framework extensions

The more ambitious, more useful in practice for future studies would be to enhance the study with a specific location and time. At the moment, the model can evaluate whether the accident is non-lethal, serious, or lethal. Future studies could calculate the probability of an accident occurring by, for example, combining the given hour, road segment and historical patterns. However, this means that more detailed information would be required, such as the exact location of the road where the accident took place, or the exact time. This study was able to get some unpredictable results, such as DecisionTree being first ranked at Phase 2 or the SMOTE not quite managing to catch up to other techniques. Therefore, an interesting approach would be to test different ways and analyse the results. For example, this study establishes the use of nominal models instead of ordinal, based on prior literature [Zhang et al. \(2018\)](#). This would confirm whether the reasoning on which the study methodology was based was correct or incorrect.

3. Group 3: Deployment and generalisation

The final future direction would recommend to test the model on newer PRF data (2024–2025) to check tuned hyperparameters and overall result. The current version was trained on data from 2017 to 2023, but a lot may have changed in the meantime, like road infrastructure, reporting practices or traffic patterns. This would verify the model’s credibility and its suitability for deployment. That could also lead to help create a decision-support tool for PRF authorities. Such a tool would help to identify high-risk highway segments, emergency infrastructure and target specific regions, which would reduce the outcome of fatal accidents.

References

- Ahmed, S., Hossain, M. A., Ray, S. K., Bhuiyan, M. M. I. & Sabuj, S. R. (2023), ‘A study on road accident prediction and contributing factors using explainable machine learning models: analysis and performance’, *Transportation Research Interdisciplinary Perspectives* **19**, 100814. **URL:** <https://www.sciencedirect.com/science/article/pii/S2590198223000611>
- Alves, P. J., Emanuel, L. & Pereira, R. H. (2021), ‘Highway concessions and road safety: Evi-

- dence from brazil', *Research in Transportation Economics* **90**, 101118.
URL: <https://www.sciencedirect.com/science/article/pii/S0739885921000901>
- Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. (2002), 'SMOTE: Synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research* **16**, 321–357.
URL: <http://dx.doi.org/10.1613/jair.953>
- Gabardo, A. M. P., Schünemann, G. A. A., Jaskowiak, P. A., Moreira, B. G. & Pfitscher, R. J. (2026), Highway to... determining fatal outcomes in traffic accidents based on police reports, in 'BRACIS 2025', pp. 230–244.
- Grinsztajn, L., Oyallon, E. & Varoquaux, G. (2022), 'Why do tree-based models still outperform deep learning on tabular data?'.
URL: <https://arxiv.org/abs/2207.08815>
- He, H., Bai, Y., Garcia, E. A. & Li, S. (2008), Adasyn: Adaptive synthetic sampling approach for imbalanced learning, in '2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)', pp. 1322–1328.
- Labib, M. F., Rifat, A. S., Hossain, M. M., Das, A. K. & Nawrine, F. (2019), Road Accident Analysis and Prediction of Accident Severity by Using Machine Learning in Bangladesh, in '2019 7th International Conference on Smart Computing & Communications (ICSCC)', pp. 1–5.
- Lee, A. S. H., Yap, L. S., Chua, H. N., Low, Y. C. & Ismail, M. A. (2021), 'A data mining approach to analyse crash injury severity level', *Journal of Engineering Science and Technology* pp. 1–14. Special Issue on ACSAT 2021.
- LippoM (2024), 'Car accidents in brazil (2017-2023)', Kaggle Dataset. Accessed: 27/03/2026.
URL: <https://www.kaggle.com/datasets/mlippo/car-accidents-in-brazil-2017-2023>
- Lundberg, S. M. & Lee, S.-I. (2017), A unified approach to interpreting model predictions, in 'Advances in Neural Information Processing Systems', Vol. 30, Curran Associates, Inc., Long Beach, CA, USA, pp. 4765–4774.
URL: <https://arxiv.org/abs/1705.07874>
- Narwane, S. & Sawarkar, S. (2019), 'Machine learning and class imbalance: A literature survey', *Industrial Engineering Journal* **12**.
- Nunes, R. & Silva, J. (2021), Traffic accident reduction potential and economic impact - automotive safety items, in 'XXIX SIMEA — Simpósio Internacional de Engenharia Automotiva', pp. 578–586.
- Scholz, J. C. W., Pinheiro, Y. P. & Silva, T. H. (2024), Predicting highway accident severity in brazil: Environmental factors and vehicle features, in '2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)', pp. 740–747.

Shwartz-Ziv, R. & Armon, A. (2022), ‘Tabular data: Deep learning is not all you need’, *Information Fusion* **81**, 84–90.

URL: <https://arxiv.org/abs/2106.03253>

Tanha, J., Abdi, Y., Samadi, N., Razzaghi, N. & Asadpour, M. (2020), ‘Boosting methods for multi-class imbalanced data classification: an experimental review’, *Journal of Big Data* **7**, 70.

Zhang, J., Li, Z., Pu, Z. & Xu, C. (2018), ‘Comparing prediction performance for crash injury severity among various machine learning and statistical methods’, *IEEE Access* **PP**, 1–1.

A Project Links

GitHub repository:	GitHub Link
DkIT Mahara:	Mahara Link
Dataset (Kaggle):	Kaggle Link
LinkedIn:	LinkedIn Link

B Tools

B.1 Development Environment

- **Operating System:** Windows 11 on Lenovo Legion Slim 5 (RTX 4060, 16 GB RAM)
- **Python distribution:** Anaconda 3, Python 3.11
- **IDE:** Visual Studio Code with Jupyter extension
- **Version control:** Git with GitHub remote
- **LaTeX editor:** Overleaf (online)

B.2 Python Libraries

- **Data handling:** pandas, numpy, pyarrow (parquet)
- **Preprocessing and metrics:** scikit-learn
- **Classifiers:** scikit-learn (Logistic Regression, Decision Tree, Random Forest), XGBoost, LightGBM, CatBoost

- **Imbalance handling:** imbalanced-learn (RandomOverSampler, RandomUnderSampler, SMOTE, ADASYN, SMOTETomek, imblearn Pipeline)
- **Hyperparameter tuning:** Optuna
- **Explainability:** SHAP (v0.51) — TreeExplainer and visualisations
- **Persistence:** joblib for model files; numpy `.npz` for compressed SHAP arrays
- **Plotting:** matplotlib, plotly (for Optuna diagnostic plots), kaleido (for plot export)

B.3 Writing and Translation Tools

- **Grammar and spell-check:** Grammarly
- **Translation and vocabulary:** Google Translate, DeepL
- **Generative AI:** Used for code assistance and concept explanation only, per DkIT policy. All AI prompts are documented in Appendix D.

C Plots

C.1 Optuna Diagnostic Plots — Non-Winning Studies

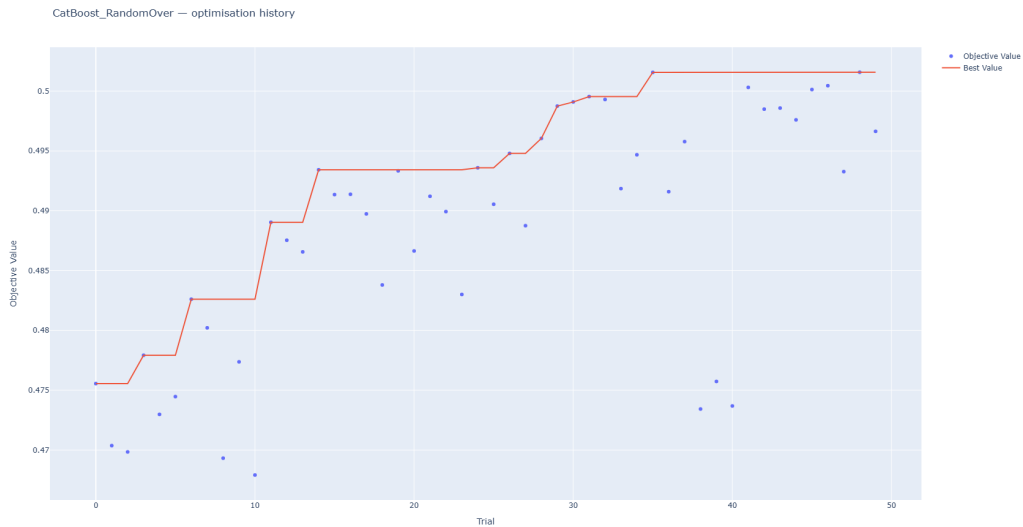


Figure 12: CatBoost RandomOver history plot



Figure 13: XGBoost ClassWeight history plot

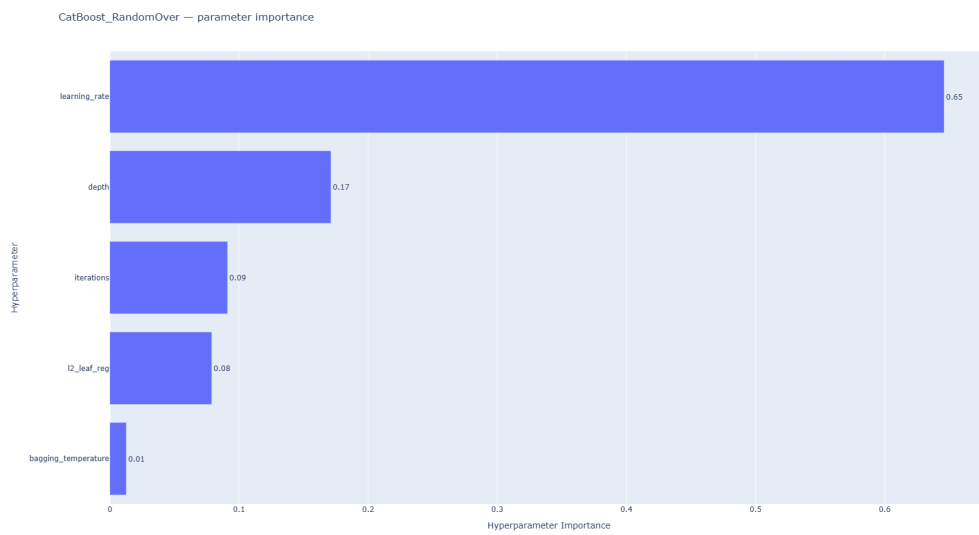


Figure 14: CatBoost RandomOver importance plot

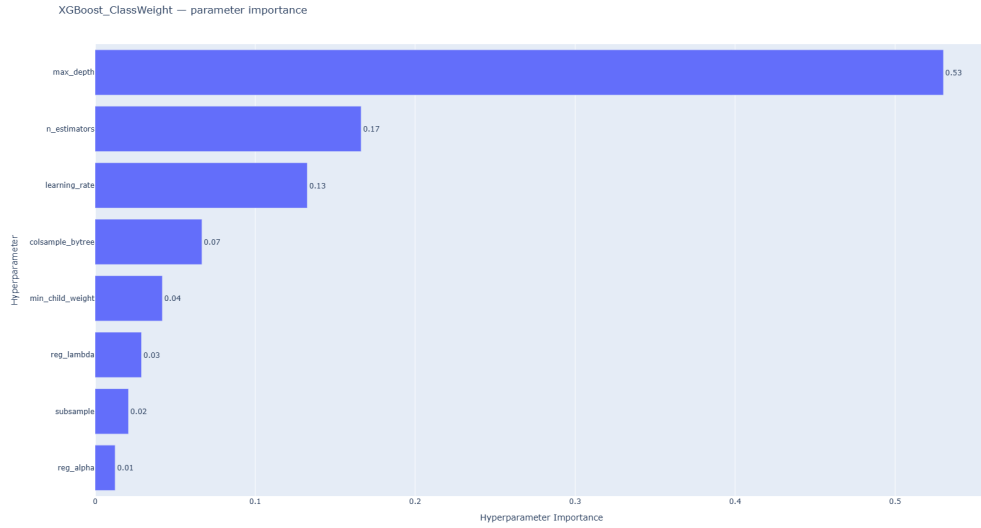


Figure 15: XGBoost ClassWeight importance plot

C.2 SHAP Plots

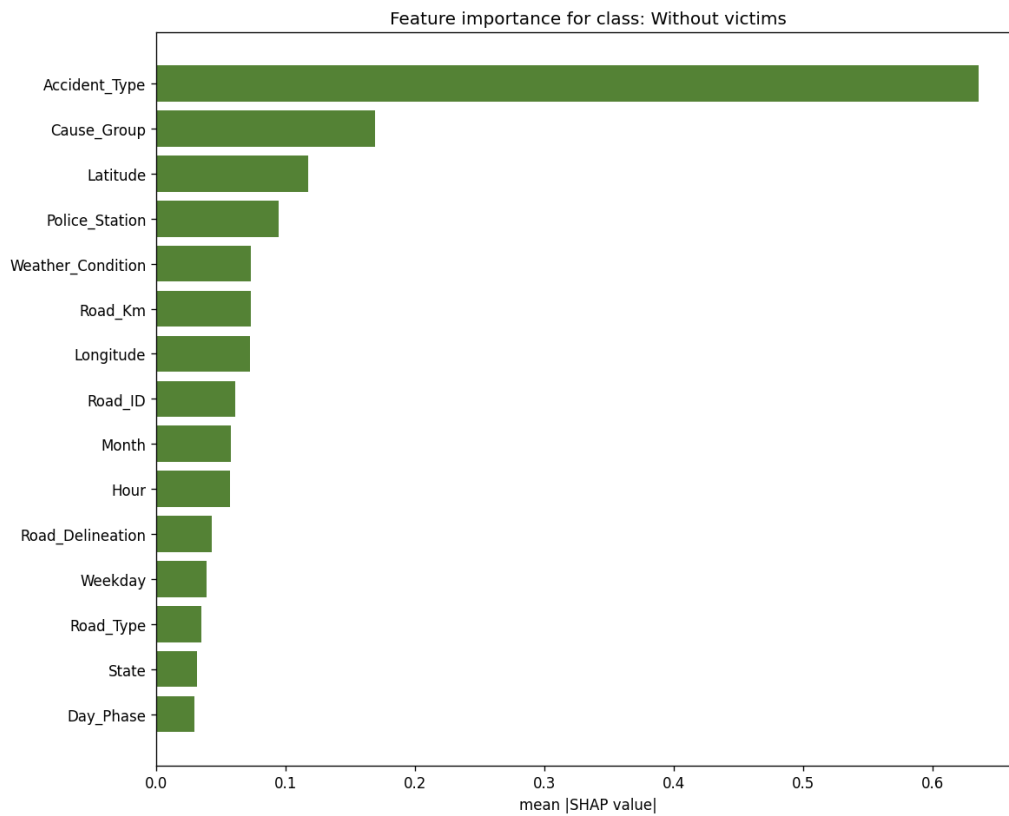


Figure 16: Per class plot - Without Victims

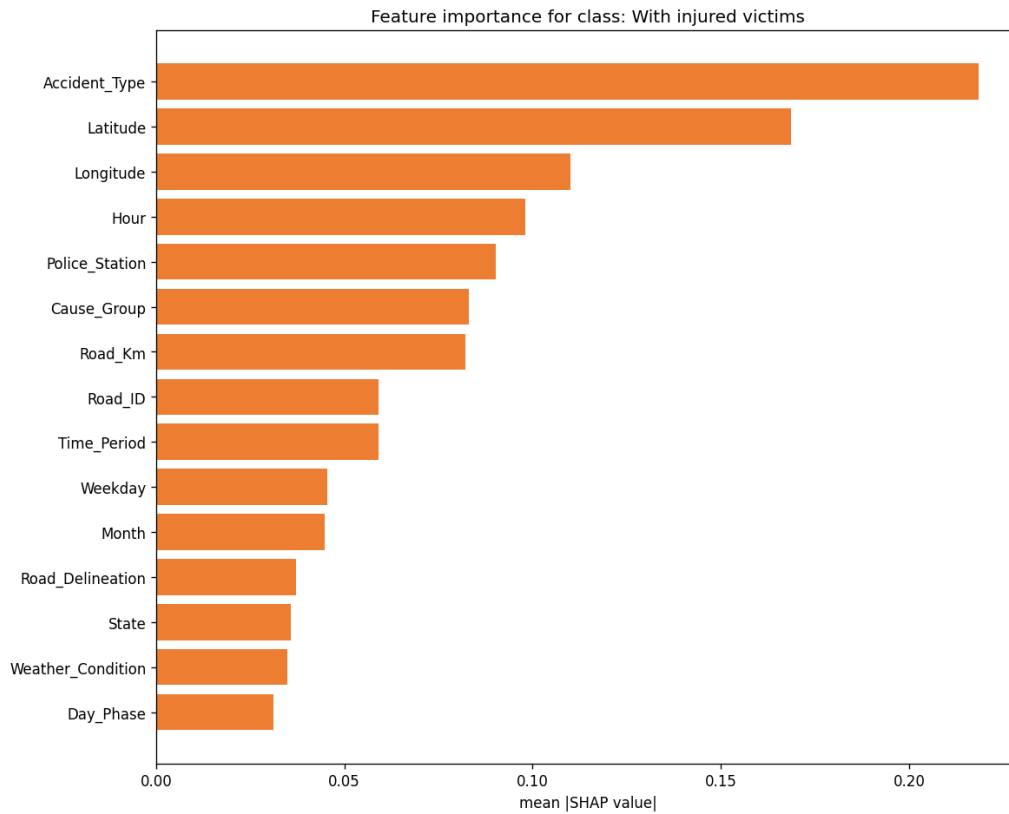


Figure 17: Per class plot - With Injured Victims

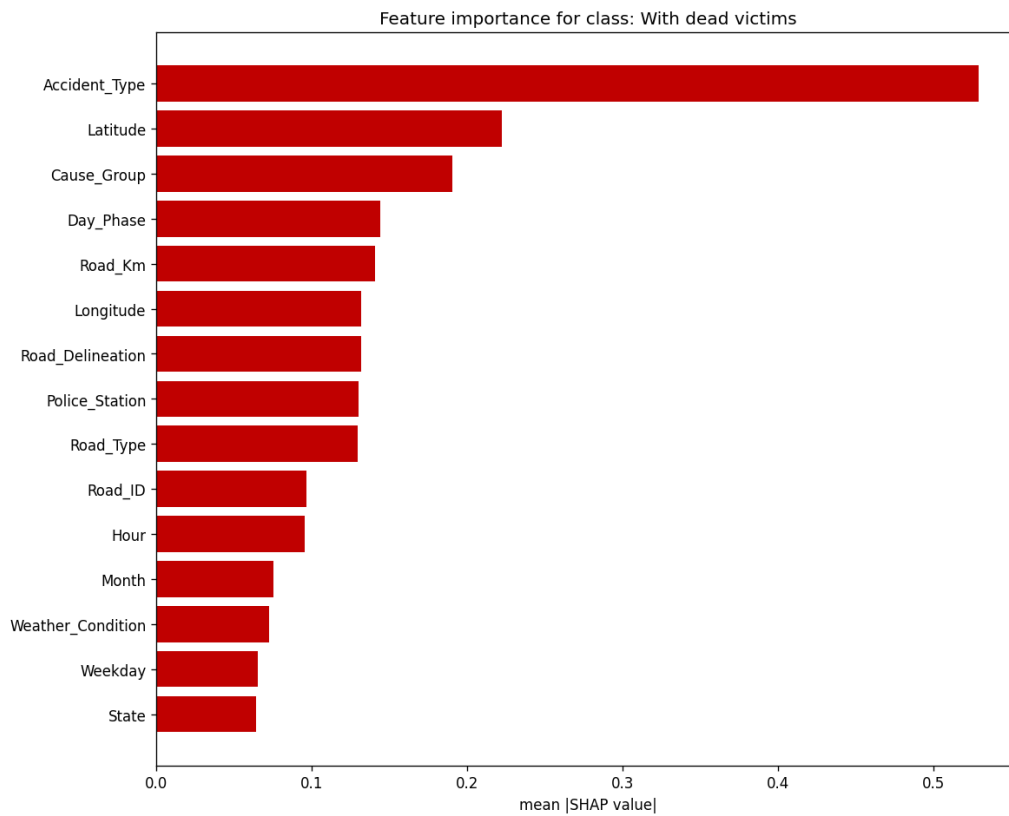


Figure 18: Per class plot - With Dead Victims

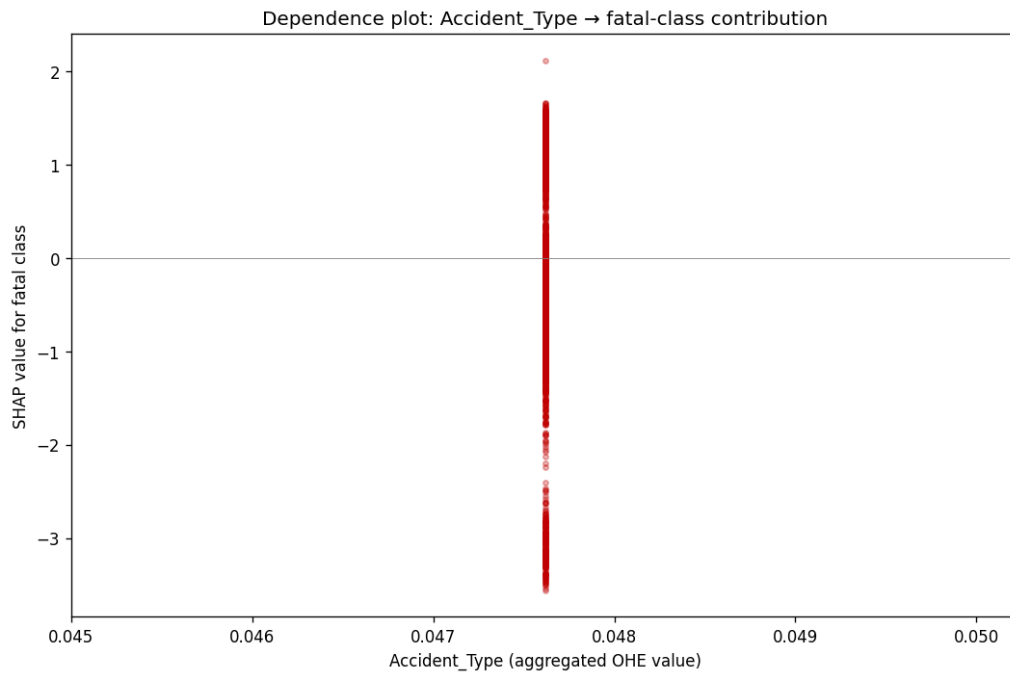


Figure 19: Dependence plot for Accident_Type

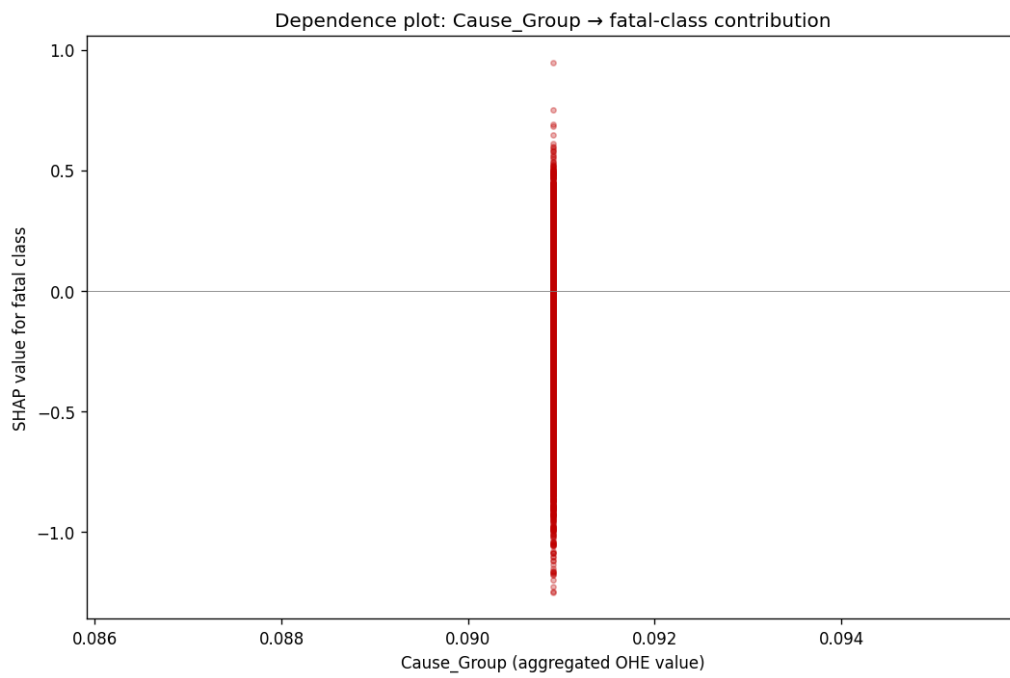


Figure 20: Dependence plot for Cause_Group

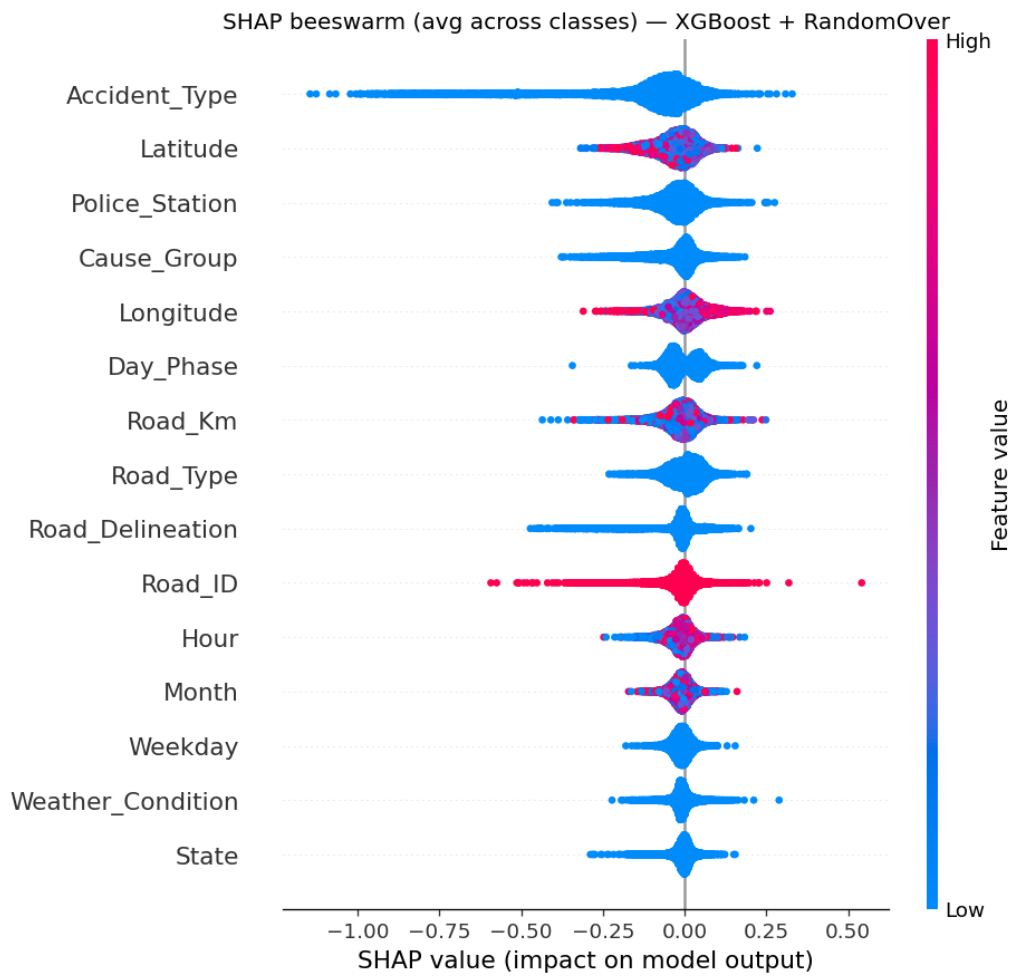


Figure 21: Global Beeswarm Plot

D Prompts for Generative AI

Prompt 1

Create a function that categorizes accident causes by keyword into fewer than 85 groups. The key is to group similar cases together to avoid duplication, and then specify the percentage of accidents that fall into each group. These groups are listed below.

Accident_Cause	
Driver's lack of attention to conveyance	107788
Incompatible velocity	43722
Driver's lack of reaction	40883
Driver broke the laws of transit	29318
Alcohol consumption	22140
Disrespect of safe distance from the next car	17929
Driver was sleeping	16820
Mechanical loss/defect of vehicle	15850
Acessing the road without seeing the presence of other vehicles	14973

Slippery track	12648
Driver failed to keep distance from the vehicle in front	12262
Alcohol ingestion by the driver	11229
Animals on the road	10533
The driver passed the next car improperly	9943
Lane change maneuver	9905
Pedestrian's lack of attention	8076
Excessive use of the car's tire	7566
Electrical or mechanical flaws	7246
Driver was in the opposite direction	5418
Road's defect	4487
Disrespecting the intersection	4106
Static object on the drainage gate	3326
Rain	3095
Cardiac attack	2951
Visibility restriction	2403
Excessive load/cargo	2364
Traffic with a motorcycle (or similar) between lanes	2290
Prohibited conversion	2266
Alcohol and/or drug ingestion by the pedestrian	2138
Pedestrian was walking in the road	1780
Unexpected pedestrian entry	1766
Irregular access	1705
Pedestrian was crossing the road outside of the crosswalk	1583
Accumulation of water on the road	1533
Driver had a cardiac attack	1515
Natural phenomena	1430
Deficiency of vehicle's sinalization/ilumination system	1258
Inadequate sinalization of the road	1193
Roads with holes without cement	1116
Abrupt use of the car's brake	1095
External fight	1084
Disobedience to laws of transit by the pedestrian	1055
Car's brake problem	846
Driving on the breakdown lane	827
Driver changed the lane illegally	778
Driver disrespected the red traffic light	741
Curvy road	726
Other flaws/problems in the road	660
Road had lots of sand/wreckage	651
Absence of sinalization	526
Sinking or ondulation in the pavement	483

Uneven breakdown lane	480
Oil accumulation on the road	437
Poor illumination (of the road)	411
Stopping at a prohibited place	403
Road works (in maintenance)	342
Drug ingestion	331
Driver using cellphone	322
Urban area without appropriate pedestrian walking	275
Obstacle in the road	238
Driver was using drugs	236
No breakdown lanes	234
Unlevel track	180
Fog	170
Car's suspension system with problems	163
Temporary return	121
Bad positioning of traffic sign posts	118
driving on the breakdown lane	114
Prominent downhill	90
Smoke	87
Visibility restrictions in horizontal curves	71
Pedestrian crosswalk with insufficient width	34
prohibited modification (of the car)	31
car's on sidewalk	31
Failed to activate the motorcycle headlight (or similar)	31
Visibility restrictions in vertical curves	30
speed reducer wasn't working correctly	25
engaging in drifting	24
Traffic lights with problems	22
unregulated headlights	20
hidden/covered sinalization (of the road)	11
robbery attempt	10
mental disorder (except suicide)	6
suicide (presumed)	5
inefficiency of the drenage system	4

Prompt 2

I got this Brazilian traffic accident data. Each row has a State column with those 2-letter codes.

State

MT 1.230
BA 1.229
MG 1.219

GO	1.217
MS	1.215
AC	1.212
MA	1.203
AP	1.194
ES	1.194
RO	1.194
RS	1.183
TO	1.179
PR	1.159
PI	1.156
PA	1.155
AL	1.152
SC	1.152
AM	1.138
RJ	1.126
PE	1.113
CE	1.111
RR	1.110
RN	1.105
SE	1.104
PB	1.101
SP	1.083
DF	1.080

Therefore for my EDA I need to group them into the 5 big regions Brazil uses (north, northeast, center-west, southeast, south). Could you also double check that all 27 states are there and nothing got put in two regions by accident.

Prompt 3

(Asked AI to create tables for Overleaf based on the tables from EDA, and personally modified them to be more suitable for the interim report. Here is the example:)

Could you create a table for Overleaf based on the output I got from my code?

	accidents	fatal_share	accident_share
Cause_Group			
Driver behaviour	227662	0.065	0.492
Speed & following distance	75854	0.058	0.164
Alcohol / Drugs	36074	0.054	0.078
Vehicle defect	34529	0.029	0.075
Road condition / infrastructure	28601	0.042	0.062
Driver state (fatigue / medical / distraction)	21614	0.094	0.047

Pedestrian-related	14569	0.309	0.031
Animals	10533	0.050	0.023
Visibility & signage	5372	0.084	0.012
Weather	4525	0.047	0.010
Other	3800	0.056	0.008

Prompt 4

Create an image of a blank map of Brazil with roads 116 and 101 highlighted (don't label them or anything like that)

Prompt 5

Based on my interim report I've decided on three things to do for phase 1: 1. stratified 80/20 split on Victims_Condition, fixed seed 2. evaluation module Accuracy, macro-F1 (primary), per-class F1 and MCC (secondary) 3. DummyClassifier baseline as Naive classifier. How should I approach with writing the evaluation function as a reusable import. Can you help me decide folder structure and tell me the differenced between parquet and CSV files.

Prompt 6

As you just mentioned the evaluation function, could you tell me exactly how would you compute accuracy, macro-F1, per-class F1, and MCC in one call.

Prompt 7

And how would you structure it?

Prompt 8

For F1-score, why would you set zero_division to be 0

Prompt 9

What is DummyClassifier exactly and what is the difference between DummyClassifier and Logistic Regression.

Prompt 10

Ok, could write the code again with all information you gain through my questions and write a code which would load the cleaned parquet file, apply the 80/20 stratified split with random_state=25, and save the four output files back to parquet.

Prompt 11

Could you also print how many rows per set is there so I can see the visible ratio?

Prompt 12

For Phase 2 I have decided to use 6 machine learning models (Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, CatBoost). Could tell me how should I approach this specific models, what should I be careful about or remember to do and compare the macro-F1 result.

Prompt 13

Preprocessing pipeline for all six classifiers, why should I do it and what benefits it contains?

Prompt 14

What is sparse_output exactly?

Prompt 15

5-fold stratified cross-validation with cross_val_score and scoring='f1_macro', how should I set it up.

Prompt 16

Why does XGBoost need integer-encoded targets for multiclass?

Prompt 17

How would you adapt XGBoost to handle multiple classes, if you know what I mean?

Prompt 18

Is it possible to write a function which would build a Pipeline with StandardScaler for numerics and OneHotEncoder(handle_unknown='ignore', sparse_output=True) for categoricals? Before using classifiers.

Prompt 19

Ok it looks good, now is it possible to write a loop, which would use this pipeline for all six classifiers and compare them in CSV file? If so, how?

Prompt 20

LightGBM is last, did I do something wrong? Is it okay or should I change something?

Prompt 21

I had finally finished the code to compare all the models but I would like to have quick sanity check for DecisionTree. Could you take this code [insert the baseline_models.ipynb] and just run it with max_depth = 20. I want to test whenever it is overfitted. Could you tell me any recommendation?

Prompt 22

Could you add those two test to it to check it?

Prompt 23

What is the difference between sklearn.pipeline.Pipeline and imblearn.pipeline.Pipeline, how is it connected with SMOTE?

Prompt 24

RandomOverSampler vs SMOTE, what is the difference?

Prompt 25

ADASYN vs SMOTE, what is the difference?

Prompt 26

How would you implement all 7 imbalance technique (None, RandomOverSampler, RandomUnderSampler, SMOTE, ADASYN, SMOTE+Tomek, Class weighting) with the top 3 models?

Prompt 27

Would you run everything at the same time or gradually? Could you time as well?

Prompt 28

What is wrong with combination XGBoost and class_weight, what should I do about it?

Prompt 29

How would you write a manual 5-fold CV loop?

Prompt 30

Also ensuring that the results are comparable with those of my other experiments

Prompt 31

How do I save models, what options I have?

Prompt 32

Could you write a code to save those files in Joblib.

Prompt 33

[Insert the results of Phase 3] DecisionTree did not performed the best, what could be the reasoning? Again, did I do something wrong? I was trying to find something on the internet, but I am still not sure.

Prompt 34

Why did SMOTE-TOMEK took 9 hours each to perform. The results are not as great to mention.

Prompt 35

For hyperparameter tuning, I want to use Bayesian hyperparameter optimisation, could you show me how would you program it with those saved Joblib files with the seed 25.

Prompt 36

I would like to run it 50 times, each of them.

Prompt 37

Would it be possible to have some history plots and importance plots?

Prompt 38

This seems alright. Any other suggestion for the code?

Prompt 39

Yea, load_if_exists=True would be usefull. Try to implement it.

Prompt 40

Yes, write a code to save it as a Joblib file.

Prompt 41

I think I have set the values for the hyperparameters correctly. Maybe only Maxdepth and Depth. Should I do something about it? or is it okay?

Prompt 42

Also from those plots, there is small difference of the final value from 30 to 50. But thats should be optimal I think. Any recommendation?

Prompt 43

Ok, almost finished. Now, I need help to write a code for explainability by using TreeSHAP. Any suggestion how should I see this approach?

Prompt 44

How would you approach the final test set evaluation? How would you write a code ofr it?

Prompt 45

How would you test it?

Prompt 46

Write a code to save the metrics DataFrame and confusion matrix to CSV files.

Prompt 47

I heard something about .npz , would you recommended for saving the shap values?

Prompt 48

Also, It would be nice if possible to convert back the OHE features. Is it even possible?

Prompt 49

Now I understand, ok so, could you write a code which would aggregate raw OHE SHAP values back to the original 22 features?

Prompt 50

For explainability, It would be nice to create some plots to better see the results. Is there any specific which would you recommend, besides standard plots?

Prompt 51

Ok could you also create a global bar, also Beeswarm plot you just mentioned for Global SHAP plots.

Prompt 52

Also could you create a for loop to create per-class shap importance plot.

Prompt 53

Could you generate three per-class bar plots for fatal accidents?

Prompt 54

How would you write a code to see the correct prediction of the model?

Prompt 55

Ok, could you do the waterfall plot as you just mention?

Prompt 56

Now, finally, now [insert the 10k sample code], I want to do the full test set for SHAP. I think it should be finished now.